

Go (Golang)

SDE2 Interview Guide

100 Questions & Answers with Code Examples

Level: Mid (2–4 yrs)	Stack: Go / Golang	Roles: SDE2 / Backend
----------------------	--------------------	-----------------------

#	Topic	Questions
1	Goroutines & Concurrency	Q1–Q15
2	Channels & Select	Q16–Q28
3	Runtime & Memory	Q29–Q40
4	Interfaces & Types	Q41–Q52
5	Error Handling	Q53–Q60
6	Context	Q61–Q67
7	Generics	Q68–Q74
8	Testing & Benchmarking	Q75–Q82
9	Go Patterns	Q83–Q92
10	Miscellaneous & Advanced	Q93–Q100

> **Focus:** Container Fundamentals, Dockerfile, Networking, Volumes, Compose, Security, Kubernetes Basics, Go Containerization | **Level:** SDE2

Table of Contents

- [Dockerfile & Images](#2-dockerfile--images) — Q21–Q40
- [Docker Networking](#3-docker-networking) — Q41–Q55
- [Volumes & Storage](#4-volumes--storage) — Q56–Q65
- [Docker Compose](#5-docker-compose) — Q66–Q75
- [Security & Production Best Practices](#6-security--production-best-practices) — Q76–Q85
- [Kubernetes Basics & Go Containerization](#7-kubernetes-basics--go-containerization) — Q86–Q100

Container Fundamentals

Q1. What is a container and how does it differ from a VM?

Difficulty: Easy

VM (Virtual Machine):

- Hypervisor virtualizes hardware
- Each VM has: full OS kernel + user space
- Size: GB (OS + app)
- Boot time: minutes
- Isolation: strong (separate kernel)
- Overhead: high (full OS per VM)

Container:

- Shares host OS kernel (via Linux namespaces + cgroups)
- Each container has: isolated user space only
- Size: MB (app + dependencies, no OS kernel)
- Boot time: milliseconds
- Isolation: process-level (same kernel)
- Overhead: minimal

	VM	Container
Kernel	Own	Shared (host)
Size	GBs	MBs
Startup	Minutes	Milliseconds
Isolation	Strong	Process-level
Use case	Full OS needed	App packaging

When to use VM:

- Different OS than host
- Maximum isolation required (security-sensitive)
- Legacy applications requiring specific kernel

When to use containers:

- Microservices
- CI/CD pipelines
- Consistent dev/prod environments
- Rapid scaling

Q2. What are Linux namespaces and cgroups?

Difficulty: Hard

Namespaces: isolate resources between processes

Each container = its own set of namespaces

pid: process IDs (container has PID 1)

```
net: network interfaces, routes, ports
mnt: mount points, filesystem view
uts: hostname and domain name
ipc: inter-process communication (shared memory, semaphores)
user: user/group IDs (UID mapping)
cgroup: cgroup root (container's resource limits)
```

```
cgroups (Control Groups): limit/account resource usage
cpu: CPU time allocation (cpu.shares, cpu.cfs_quota_us)
memory: memory limit + OOM killer
blkio: block I/O bandwidth limits
network: network bandwidth (via tc, not native cgroup)
```

Result:

```
Container process sees: its own PID space, its own network, its own filesystem
But: shares host kernel, same hardware
```

```
Container engine (Docker/containerd):
Creates namespaces: unshare(2) syscall
Sets up cgroups: write to /sys/fs/cgroup/
Sets up overlay filesystem
Runs container process with namespaces
```

Check container cgroups:

```
cat /sys/fs/cgroup/memory/docker/{container_id}/memory.limit_in_bytes
```

Q3. What is Docker architecture?

Difficulty:
Medium

Docker architecture:

```
[Docker CLI] → [Docker Daemon (dockerd)] → [containerd] → [runc]
```

```
Docker CLI: command-line tool (docker run, build, push)
Docker Daemon (dockerd): manages containers, images, networks, volumes
REST API on /var/run/docker.sock (Unix socket)
```

```
containerd: container runtime (pulls images, manages containers)
OCI-compliant runtime interface
```

```
runc: low-level runtime (creates Linux namespaces, cgroups, starts process)
OCI runtime spec implementation
```

```
Image Registry: Docker Hub, ECR, GCR, Harbor
Store and distribute images
```

Flow for "docker run nginx":

1. CLI → API → dockerd
2. dockerd → containerd: "run image nginx"
3. containerd: pull image from registry if not cached
4. containerd → runc: create container
5. runc: create namespaces, cgroups, overlay FS, start /bin/nginx

Alternative runtimes:

```
podman: rootless, daemonless (compatible with Docker CLI)
containerd: used by Kubernetes directly (no Docker daemon needed)
cri-o: lightweight for Kubernetes
```

Q4. What is a Docker image?

Difficulty: Easy

Image: read-only template for creating containers
 Consists of: ordered stack of layers
 Layer: set of filesystem changes (files added/modified/deleted)

Image format (OCI / Docker):
 Manifest: JSON describing layers + config
 Layers: tar archives of filesystem diffs
 Config: entrypoint, env vars, labels, etc.

Layered storage (Union FS):
 Base layer: FROM ubuntu:22.04 → ubuntu filesystem
 Layer 2: RUN apt-get install nginx → nginx added
 Layer 3: COPY app.conf /etc/nginx/ → conf added
 Container: +writable layer on top (deleted when container removed)

`docker image inspect nginx:latest:`
 Layers: [sha256:abc..., sha256:def..., sha256:ghi...]
 Config: {Cmd: [nginx, -g, daemon off;], ExposedPorts: {80/tcp: {}}}

Layer caching:
 Layers shared between images (same hash = same content)
 ubuntu base: shared between all Ubuntu-based images
 npm install layer: shared if package.json unchanged
 → saves disk space, speeds up pulls

`docker history nginx:`
 Shows each layer, command that created it, size

Q5. What is the difference between an image and a container?

Difficulty: Easy

Image: static snapshot (blueprint)
 Read-only
 Can be stored in registry
 Can be shared
 One image → many containers

Container: running instance of an image
 Image layers (read-only) + writable layer on top
 Has own filesystem state, network, processes
 Ephemeral: stopped container still exists (not running)

States:
 Created: container created, not started
 Running: container process is active
 Paused: process suspended (cgroup freezer)
 Stopped/Exited: process terminated, writable layer still exists
 Removed: writable layer deleted

Commands:

<code>docker create nginx</code>	→ creates container (Created state)
<code>docker start <id></code>	→ starts it (Running)
<code>docker stop <id></code>	→ stops it (Exited) – SIGTERM then SIGKILL after timeout
<code>docker kill <id></code>	→ immediate kill (SIGKILL)
<code>docker rm <id></code>	→ removes container

`docker rmi nginx:latest` → removes image (fails if containers using it)

Container ID: 12-char hex (from 64-char full hash)

`docker ps -a` → all containers (including stopped)

`docker ps` → running containers only

Q6. What is the Docker build process?

Difficulty:
Medium

`docker build -t myapp:v1 .`

Build context: directory (.) sent to Docker daemon

All files in context are available to COPY/ADD instructions

.dockerignore: exclude unnecessary files (like .git, node_modules)

Build steps:

1. Read Dockerfile
2. Send context to daemon
3. Execute each instruction, creating image layers
4. Cache each layer by hash of instruction + parent layer

Build cache:

If instruction unchanged AND parent layer unchanged → use cache

Cache invalidated: change in instruction or file it references

dockerfile instruction cache busting:

`RUN apt-get update` ← cached (instruction unchanged)

`COPY . /app` ← cache bust if ANY file changes

`RUN go build` ← always runs after COPY cache bust

Buildkit (modern, default in Docker 23+):

Parallel execution of independent build stages

Secrets and SSH agent (no secrets in layers)

Inline caching: `--cache-from`

Multi-platform: `--platform linux/amd64,linux/arm64`

`docker build \`

`--platform linux/amd64 \`

`--cache-from type=registry,ref=myapp:cache \`

`-t myapp:v1 .`

Q7. What is a container registry?

Difficulty: Easy

Registry: service for storing and distributing Docker images

Public registries:

Docker Hub: `docker.io/library/nginx:latest`

GitHub Container Registry: `ghcr.io/org/image:tag`

Quay.io: `quay.io/org/image:tag`

Cloud-managed registries:

AWS ECR: `123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:v1`

GCP Artifact Registry: `us-docker.pkg.dev/project/repo/image:tag`

Azure Container Registry: `myacr.azurecr.io/image:tag`

Self-hosted:

Harbor: enterprise-grade, security scanning, RBAC

Nexus Repository: for enterprise artifact management
 Docker Registry (OSS): simple self-hosted

Image naming: registry/namespace/image:tag
 nginx:latest → docker.io/library/nginx:latest
 myorg/myapp:v1.2.3 → docker.io/myorg/myapp:v1.2.3

Tags: mutable (can point to different image)
 latest: convention for newest, not guaranteed freshness
 Digest: immutable: sha256:abc123... → always same image
 Best practice: use specific tag (v1.2.3) or digest in production

Authentication:
 docker login registry.example.com
 ECR: aws ecr get-login-password | docker login --username AWS ...
 GCR: gcloud auth configure-docker

Q8. What is container orchestration?

Difficulty: Easy

Orchestration: automated management of containers at scale
 Scheduling: which node runs which container
 Scaling: add/remove containers based on load
 Service discovery: how containers find each other
 Load balancing: distribute traffic across container instances
 Health management: restart failed containers
 Rolling updates: deploy new versions without downtime
 Storage: provision persistent storage for stateful apps

Kubernetes (K8s): dominant orchestrator
 Pod: smallest deployable unit (1+ containers sharing network/storage)
 Deployment: manages replicated pods
 Service: stable network endpoint for pods
 Ingress: HTTP(S) routing to services

Docker Swarm: simpler orchestration built into Docker
 Easier to set up than K8s
 Less powerful (no autoscaling, less mature ecosystem)

Amazon ECS: AWS-native container service
 Fargate: serverless (no EC2 management)
 EC2 launch type: control underlying instances

Why orchestration matters:
 Containers by themselves: no HA, no scaling, no discovery
 Orchestrator: production-grade container management
 K8s vs ECS: K8s = more control + more complexity; ECS = easier AWS integration

Q9. What is the OCI (Open Container Initiative)?

Difficulty:
 Medium

OCI: open standards for containers
 Prevents vendor lock-in
 Ensures compatibility between runtimes and tools

Specifications:
 OCI Image Spec: format for container images (layers + manifest + config)
 OCI Runtime Spec: what a container runtime must do (low-level spec)

OCI Distribution Spec: API for pushing/pulling images (registry API)

Implementations:

Docker: OCI-compatible (uses containerd + runc)
 Podman: OCI-compatible, rootless, daemonless
 containerd: OCI-compliant runtime
 runc: OCI Runtime reference implementation
 cri-o: OCI-based runtime for Kubernetes

Compliance means:

Images built with Docker can run on containerd, podman
 Any OCI runtime can run OCI images
 Kubernetes can use any OCI-compatible runtime via CRI

Benefits:

Build once, run anywhere (any OCI runtime)
 Switch from Docker to Podman without rebuilding images
 Cloud providers support OCI images natively

Q10. What is overlay filesystem?

Difficulty: Hard

OverlayFS: union filesystem implementation in Linux
 Layers multiple directories into single merged view
 Used by Docker for efficient image storage

Layers:

lowerdir: read-only layers (image layers)
 upperdir: writable layer (container-specific changes)
 workdir: OverlayFS internal working directory
 merged: combined view (container sees this)

Example:

Image layers: [ubuntu FS] + [nginx binaries] + [nginx config]
 Container upper: (empty initially)
 Merged: ubuntu + nginx + config (appears as one FS)

Container writes /var/log/nginx/access.log:

→ File created in upperdir only
 → Not in lowerdir (image unchanged)

Container deletes /etc/nginx/nginx.conf:

→ Whiteout file created in upperdir
 → Lower layer file "hidden"

Container modifies /etc/nginx/nginx.conf:

→ Copy-on-write: file copied to upperdir, then modified
 → Image file unchanged

Storage drivers:

overlay2: default (recommended, Linux kernel 4.0+)
 aufs: older, not in mainline kernel
 devicemapper: block-level (slower, complex)
 btrfs, zfs: filesystem-level (requires specific FS)

Q11. ### Q11-Q20: More Container Fundamentals

Difficulty:
Medium

Q	Topic
Q11	Container lifecycle: create, start, stop, remove
Q12	docker exec vs docker attach: when to use each
Q13	docker logs: streaming and filtering container output
Q14	Container resource limits: --memory, --cpus flags
Q15	docker stats: real-time container resource monitoring
Q16	Ephemeral vs persistent containers
Q17	Multi-stage build benefits and use cases
Q18	Docker content trust and image signing
Q19	Container escape and kernel vulnerabilities
Q20	Rootless containers: benefits and limitations

Dockerfile & Images

Q12. What are the most important Dockerfile instructions?

Difficulty: Easy

```
FROM golang:1.22-alpine AS builder
# Base image: source of your OS and runtime environment
# Use specific tags, never FROM latest in production

WORKDIR /app
# Set working directory inside container
# Creates if doesn't exist

COPY go.mod go.sum ./
# COPY <src> <dst>
# src: relative to build context
# dst: inside container

RUN go mod download
# Execute shell command, creates new layer
# Use && to chain and minimize layers

COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o /server ./cmd/server

# Second stage: minimal runtime image
FROM alpine:3.19

# Metadata labels (OCI annotations)
LABEL org.opencontainers.image.source="https://github.com/myorg/myapp"
LABEL org.opencontainers.image.version="1.2.3"

WORKDIR /app
# Copy only the binary from builder stage
```



```
COPY --from=builder /server .

# Expose documents the port (doesn't actually expose)
EXPOSE 8080

# ENTRYPOINT: non-overridable executable
# CMD: default arguments (overridable with docker run args)
ENTRYPOINT ["/app/server"]
CMD ["--port", "8080"]

# USER: run as non-root (security best practice)
USER 1000:1000
```

Q13. What is a multi-stage build?

Difficulty:
Medium

```
# Stage 1: Builder (large image with build tools)
FROM golang:1.22-alpine AS builder
WORKDIR /build
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -trimpath -o /bin/server ./cmd/server

# Stage 2: Test runner (optional)
FROM builder AS tester
RUN go test ./...

# Stage 3: Minimal runtime (tiny image, no build tools)
FROM scratch AS runtime
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
COPY --from=builder /bin/server /server
EXPOSE 8080
ENTRYPOINT ["/server"]

# Build only final stage (default)
docker build -t myapp:v1 .

# Build up to specific stage
docker build --target tester -t myapp:test .

# Size comparison:
# golang:1.22-alpine: ~250MB
# Final scratch image: ~8MB (binary + CA certs only)
# Final alpine image: ~12MB

Benefits:
  Minimal attack surface (no compiler, shell, pkg manager in prod image)
  Smaller images: faster pull, less storage, less network
  Clear separation: build deps vs runtime deps
  Cannot exfiltrate build secrets from final image
```

Q14. What is ENTRYPOINT vs CMD?

Difficulty:
Medium

```
# CMD: default command, easily overridden
FROM ubuntu
CMD ["echo", "hello"]
```

```
# docker run myimage          → echo hello
# docker run myimage ls -la   → ls -la (CMD replaced entirely)

# ENTRYPOINT: fixed executable
FROM ubuntu
ENTRYPOINT ["echo"]
CMD ["hello"]
# docker run myimage          → echo hello
# docker run myimage world    → echo world (CMD replaced, ENTRYPOINT stays)
# docker run --entrypoint ls myimage -la → ls -la (ENTRYPOINT overridden with --entrypoint)

# Practical Go example:
ENTRYPOINT ["/app/server"]
CMD ["--config", "/etc/app/config.yaml"]
# docker run myapp          → /app/server --config /etc/app/config.yaml
# docker run myapp --port=9090 → /app/server --port=9090 (CMD replaced)

# Shell vs Exec form:
# Shell form: CMD echo hello → /bin/sh -c "echo hello"
# PID 1 = /bin/sh (not your app) → signals not forwarded!
#
# Exec form: CMD ["echo", "hello"] → runs directly
# PID 1 = echo → signals forwarded correctly
# ALWAYS use exec form for ENTRYPOINT and CMD

# Signal handling in Go:
# Process must handle SIGTERM (sent by docker stop)
# docker stop: SIGTERM → wait 10s → SIGKILL
```

Q15. What is .dockerignore?

Difficulty: Easy

```
.dockerignore: exclude files from build context
  Reduces context size (faster builds)
  Prevents sensitive files from reaching build
  Prevents cache invalidation from irrelevant files
```

Example .dockerignore for Go:

```
# Version control
.git
.gitignore

# Go build artifacts
vendor/      # if using go mod download instead

# Development files
.env
.env.local
*.env

# Test files (if not needed in image)
**/*_test.go

# Documentation
docs/
README.md

# CI/CD
```

```
.github/
Dockerfile
docker-compose*.yaml
```

```
# IDE
.idea/
.vscode/
```

Impact:

Without `.dockerignore`: context = entire repo (could be 100MB)

With `.dockerignore`: context = only needed files (few MB)

Each build: entire context sent to daemon over socket

Large context = slow even if files not COPYed (still transferred)

`.git` alone: can be 50MB+ in large repos

Q16. What is image layer caching and how to optimize it?

Difficulty:
Medium

```
# BAD: cache bust every time any file changes
COPY . .
RUN go mod download # re-runs even if go.mod unchanged

# GOOD: separate dependencies from source
# go.mod changes rarely → stable cache layer
COPY go.mod go.sum ./
RUN go mod download # only re-runs when go.mod/go.sum change

COPY . . # now bust cache for source changes only
RUN go build ...

# Python equivalent:
COPY requirements.txt .
RUN pip install -r requirements.txt # cache: stable layer
COPY . . # cache: bust on code change
RUN python setup.py build

# Node.js equivalent:
COPY package.json package-lock.json ./
RUN npm ci # cache: stable
COPY . . # cache: bust on code change
```

Cache rules:

Each instruction: `hash(instruction + parent_layer_hash)`

If hash unchanged → use cache

Once cache busted: all subsequent layers re-run

Instructions that bust cache:

- `RUN apt-get update` (content changes even if command same)
Fix: `RUN apt-get update && apt-get install -y nginx` (combine)
- `COPY . .` (changes when any file changes)
Fix: `COPY` specific files before general `COPY`
- `ARG` (if arg changes)

Q17. What are Dockerfile best practices for production?

Difficulty:
Medium

```
# 1. Use specific base image tags (not latest)
FROM golang:1.22.4-alpine3.19 AS builder

# 2. Use minimal base images
#   scratch: empty (0MB) - good for static Go binaries
#   distroless: minimal (no shell, package manager)
#   alpine: ~5MB with /bin/sh
FROM gcr.io/distroless/static:nonroot AS runtime

# 3. Run as non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser:appgroup

# 4. Minimize layers
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        ca-certificates \
        curl && \
    rm -rf /var/lib/apt/lists/*

# 5. Use COPY not ADD (unless you need tar extraction or URL)
COPY app.tar.gz /app/ # ADD auto-extracts, confusing
ADD app.tar.gz /app/  # ADD is ok here (intentional extraction)

# 6. No secrets in Dockerfile or layers
# NEVER: ENV DB_PASSWORD=secret
# Use: --secret, environment injection at runtime

# 7. HEALTHCHECK
HEALTHCHECK --interval=30s --timeout=10s --start-period=30s --retries=3 \
    CMD wget -q -O- http://localhost:8080/health || exit 1

# 8. Set metadata
LABEL org.opencontainers.image.source="https://github.com/org/repo"
LABEL org.opencontainers.image.revision="${GIT_COMMIT}"
```

Q18. What is a distroless image?

Difficulty:
Medium

Distroless: minimal container images with no OS package manager, shell, or utilities
 Contains: app + runtime dependencies only
 No: bash, sh, apt, yum, package manager, curl, wget

Maintained by: Google (gcr.io/distroless)

Types:

distroless/static:	C/C++ static binaries, Go static binaries (no libc)
distroless/base:	libc only
distroless/cc:	libc + C++ stdlib
distroless/java:	JRE
distroless/python:	Python runtime
distroless/nodejs:	Node.js runtime

Size comparison:

```
ubuntu:22.04:      77MB
debian:slim:      80MB
alpine:3.19:      7MB
gcr.io/distroless/static: 2MB
scratch (empty):  0MB + your binary
```

Benefits:

```
Minimal attack surface (no shell = no shell injection)
Smaller image = faster pull, less CVE exposure
Compliance: fewer packages = fewer CVEs to patch
```

Debugging limitation:

```
No shell → no docker exec -it bash
Solution: distroless :debug tag (adds busybox shell)
docker run --entrypoint=sh gcr.io/distroless/static:debug
```

Q19. What is Docker BuildKit and its advantages?

Difficulty: Hard

BuildKit: next-generation Docker build engine (default in Docker 23+)

Features:

1. Parallel build stages:

```
Independent stages build in parallel (vs sequential in classic)
FROM builder1 AS a
FROM builder2 AS b          # builds in parallel with a
FROM scratch
COPY --from=a ...
COPY --from=b ...
```

2. Better cache management:

```
--cache-from: import cache from remote (registry)
--cache-to: export cache to remote
BuildKit cache mounts: RUN --mount=type=cache,target=/root/.cache/go/pkg/mod
```

3. Secrets (never baked into image layers):

```
# Build command:
docker build --secret id=mysecret,src=./secret.txt .
# Dockerfile:
RUN --mount=type=secret,id=mysecret cat /run/secrets/mysecret
```

4. SSH forwarding:

```
docker build --ssh default .
RUN --mount=type=ssh git clone git@github.com:private/repo.git
```

5. Multi-platform builds:

```
docker buildx build --platform linux/amd64,linux/arm64 -t myapp:v1 .
```

6. Improved output (progress reporting, error messages)

Enable:

```
DOCKER_BUILDKIT=1 docker build . (old)
or: Docker 23+ (automatic)
```

Q20. What is Docker image scanning?

Difficulty: Medium

Image scanning: check for known CVEs in image packages

When to scan:

- In CI/CD pipeline after build (block if critical CVEs)
- Before deploying to production
- Continuously for deployed images (new CVEs discovered daily)

Tools:

- docker scout: built into Docker Desktop
- Trivy (Aqua): open source, most popular
- Snyk: commercial, integrates with GitHub
- Clair: CoreOS/Red Hat, powers Quay.io
- AWS ECR scanning: automatic scanning on push
- GitHub Container Scanning: in GitHub Actions

Trivy example:

```
trivy image myapp:v1
# Shows: CVE-ID, Severity, Package, Installed Version, Fixed Version
```

Severity levels:

- CRITICAL: patch immediately
- HIGH: patch within 7 days
- MEDIUM: patch within 30 days
- LOW: informational

Automation in CI:

```
trivy image --exit-code 1 --severity CRITICAL myapp:v1
# Exit code 1 if CRITICAL CVEs found → fail the build
```

SBOM (Software Bill of Materials):

- Manifest of all packages in image
- docker sbom myapp:v1
- Used for supply chain security compliance
- SLSA framework for build provenance

Q21. How do you tag images for production?

Difficulty: Easy

Image tagging strategy:

Semantic versioning (recommended):

- myapp:1.0.0 → stable release
- myapp:1.0.1 → patch release
- myapp:1.1.0 → minor release
- myapp:2.0.0 → major release

Git-based tagging:

- myapp:abc1234 → git commit SHA (immutable, traceable)
- myapp:main-abc1234 → branch + commit

CI/CD pipeline pattern:

- On PR: myapp:pr-42-abc1234 (test builds)
- On merge to main: myapp:main-abc1234 + myapp:latest
- On tag: myapp:v1.2.3 + myapp:1.2 + myapp:1 + myapp:latest

Docker tagging commands:

- docker tag myapp:abc1234 registry.example.com/myapp:v1.2.3
- docker tag myapp:abc1234 registry.example.com/myapp:latest

```
Immutable tags: sha256 digest
docker pull myapp@sha256:abc123... # always same image
Use in production manifests for reproducibility:
image: myapp@sha256:abc123... # Kubernetes pod spec
```

```
Avoid:
:latest in production (unpredictable)
Overwriting existing version tags (breaks reproducibility)
```

```
// Automatically tag in CI (GitHub Actions):
docker build -t myapp:${{ github.sha }} .
docker push myapp:${{ github.sha }}
docker tag myapp:${{ github.sha }} myapp:latest
docker push myapp:latest
```

Q22. ### Q31-Q40: More Image Topics		Difficulty: Medium
Q	Topic	
Q31	COPY --chown vs RUN chown: ownership in images	
Q32	Docker layer size optimization techniques	
Q33	Base image selection: ubuntu vs debian vs alpine vs distroless	
Q34	Go-specific: CGO_ENABLED=0 and static binaries	
Q35	ARG vs ENV: build-time vs runtime variables	
Q36	ONBUILD instruction for base images	
Q37	Docker image manifest and multi-arch manifests	
Q38	Image flattening: squashing layers for distribution	
Q39	Docker registry mirror configuration	
Q40	Reproducible builds: deterministic image content	

Docker Networking

Q23. What are Docker network types?	Difficulty: Medium
-------------------------------------	-----------------------

```
bridge (default):
  Software bridge on host (docker0)
  Containers get private IPs (172.17.0.x)
  Can communicate via bridge; NAT to reach host/internet
  Isolated from host network
  Use: single-host container communication
```

```
host:
  Container shares host network namespace
  No network isolation
  Port binding: same ports as host (no -p mapping needed)
```

Better performance (no NAT overhead)
Use: performance-critical, monitoring tools

none:
No networking (loopback only)
Use: batch jobs, offline processing, maximum isolation

overlay:
Multi-host networking (Docker Swarm / Kubernetes)
VXLAN tunnel between hosts
Containers on different hosts communicate as if on same network
Use: distributed applications

macvlan:
Container gets its own MAC address on physical network
Appears as physical device on network
Use: legacy apps requiring direct layer-2 access

Custom bridge networks:
docker network create --driver bridge mynet
Containers on same custom network: automatic DNS by name
vs default bridge: must use IP or --link (deprecated)

Q24. How does Docker container DNS work?

Difficulty:
Medium

Docker DNS:
Each container: /etc/resolv.conf → Docker DNS server (127.0.0.11)
Docker embedded DNS: resolves container names to IPs

Custom bridge networks: automatic DNS by container name
docker network create mynet
docker run --name db --network mynet postgres
docker run --name app --network mynet myapp

Inside app container: ping db → resolves to db container IP
Inside app container: curl http://db:5432 → works!

This is how docker-compose works: services communicate by service name

Default bridge network: NO automatic DNS
Must use --link (deprecated) or IP addresses
Or: docker inspect to get IP

Docker Compose service discovery:
services:
 app:
 environment:
 - DB_HOST=db # service name = hostname in custom network
 db:
 image: postgres

Network aliases:
docker run --network mynet --network-alias db1 --name db postgres
Multiple aliases for same container

DNS options in custom networks:


```
--dns 8.8.8.8: use specific DNS server
--dns-search example.com: search domain
```

Q25. How does port mapping work in Docker?

Difficulty: Easy

Port mapping: publish container port to host

```
-p host_port:container_port
-p 8080:80          → host:8080 → container:80
-p 127.0.0.1:8080:80 → only localhost (not all interfaces)
-p 0.0.0.0:8080:80  → all interfaces (default)
-p 8080:80/tcp      → TCP only
-p 8080:80/udp      → UDP only
```

Implementation (iptables):

Docker adds iptables DNAT rule:

```
-A DOCKER -p tcp -m tcp --dport 8080 -j DNAT --to-destination 172.17.0.2:80
```

Random host port:

```
-p 80          → Docker assigns random available port
docker port <container> 80 → see assigned port
```

EXPOSE instruction (documentation only):

Does NOT publish port

Just documents what port the container listens on

docker run -P (uppercase): publishes all EXPOSEd ports to random host ports

Kubernetes equivalent:

Service: ClusterIP (cluster-internal), NodePort, LoadBalancer

No direct port mapping like Docker (different model)

Performance:

iptables NAT has overhead for high-throughput

host networking: no port mapping overhead (shares host ports)

--publish-all with host network: not supported (meaningless)

Q26. What is Docker network isolation?

Difficulty: Medium

Network isolation: containers on different networks cannot communicate

Default behavior:

Containers on default bridge: all can communicate

Containers on custom networks: only same-network containers communicate

Different networks: isolated by default (no routing between them)

Production pattern (defense in depth):

frontend network: nginx ↔ app servers

backend network: app servers ↔ database, redis

App servers: connected to BOTH networks (bridging)

Database: ONLY on backend network (not accessible from frontend)

docker-compose network example:

services:

nginx:

networks: [frontend]

```

app:
  networks: [frontend, backend] # bridge between networks
postgres:
  networks: [backend]           # isolated from frontend
redis:
  networks: [backend]           # isolated from frontend
networks:
  frontend:
  backend:

```

Explicit isolation benefits:

Database breach cannot reach nginx directly

Network policies enforced at infrastructure level (not just app code)

Connect existing container to additional network:

`docker network connect backend myapp`

`docker network disconnect frontend mydb`

Q27. ### Q45-Q55: More Networking Topics

Difficulty:
Medium

Q	Topic
Q45	Docker ingress networking in Swarm mode
Q46	Container IP address assignment and inspection
Q47	Network performance: bridge vs host vs macvlan
Q48	Exposing internal services with port mapping security
Q49	Docker network debugging: tcpdump, netstat inside containers
Q50	Service mesh (Istio) vs Docker networking
Q51	IPv6 support in Docker networks
Q52	Docker network plugins (Calico, Weave, Flannel)
Q53	Container hostname and DNS configuration
Q54	Connecting containers across Docker Compose projects
Q55	Docker network for production: security recommendations

Volumes & Storage

Q28. What are Docker volumes?

Difficulty: Easy

Problem: container filesystem is ephemeral (lost when container removed)

Solution: volumes persist data outside container lifecycle

Types:

Named Volumes (recommended):

```
docker volume create mydata
docker run -v mydata:/app/data myapp
Stored: /var/lib/docker/volumes/mydata/_data
Managed by Docker, portable, easy to backup
```

Bind Mounts:

```
docker run -v /host/path:/container/path myapp
Host directory → container path
Use: development (live code reload), sharing files with host
```

tmpfs Mounts:

```
docker run --tmpfs /tmp:rw,size=100m myapp
Stored in host RAM only (not on disk)
Use: temporary files, sensitive data that shouldn't persist
```

Anonymous Volumes:

```
docker run -v /app/data myapp
Docker creates volume with random ID
Removed with container if --rm flag used
Avoid in production (hard to manage)
```

Named volumes benefits:

```
Platform-independent (works on Linux/Mac/Windows)
Managed by Docker daemon
Easy to: docker volume inspect, backup, migrate
```

```
docker volume ls
docker volume inspect mydata
docker volume rm mydata
```

Q29. What is the difference between volumes and bind mounts?

Difficulty:
Medium

Named Volume:	Bind Mount:
Docker manages location	Host controls location
/var/lib/docker/volumes/.../_data	/any/host/path
Platform-independent	Host-path-dependent
Shared by multiple containers easily	Must specify exact host path
Good for production data	Good for development
Backup: docker run + cp	Backup: cp host path

When to use bind mounts:

```
Development: mount source code → edit without rebuild
docker run -v $(pwd):/app myapp
```

Configuration files: inject runtime config

```
docker run -v /etc/myapp/config.yaml:/app/config.yaml:ro myapp
```

Access host files in container

```
docker run -v /var/log/host:/var/log/host:ro myapp
```

When to use named volumes:

```
Persistent application data (databases, file uploads)
Production deployments
Sharing data between containers
```

```
:ro flag = read-only mount
```

```
docker run -v myconfig:/config:ro myapp
Container cannot write to /config → security benefit
```

Kubernetes equivalent:

```
PersistentVolume + PersistentVolumeClaim (named volume)
hostPath (bind mount – avoid in production, security risk)
ConfigMap / Secret (configuration injection)
```

Q30. How do you back up and restore Docker volumes?

Difficulty:
Medium

```
# Backup volume to tar archive
docker run --rm \
  -v mydata:/data:ro \
  -v $(pwd):/backup \
  alpine tar czf /backup/mydata-$(date +%Y%m%d).tar.gz -C /data .

# Restore from archive
docker volume create mydata-restored
docker run --rm \
  -v mydata-restored:/data \
  -v $(pwd):/backup \
  alpine tar xzf /backup/mydata-20240101.tar.gz -C /data

# Copy between environments
# Export:
docker run --rm -v mydata:/data alpine tar c -C /data . | \
  ssh user@remote "docker volume create mydata && \
  docker run --rm -i -v mydata:/data alpine tar x -C /data"

# PostgreSQL volume backup:
docker exec postgres pg_dump -U postgres mydb | gzip > backup.sql.gz

# Restore PostgreSQL:
gunzip -c backup.sql.gz | docker exec -i postgres psql -U postgres mydb

# Kubernetes volume backup (Velero):
velero backup create mybackup --include-namespaces myapp
velero restore create --from-backup mybackup
```

Q31. ### Q59-Q65: More Volume Topics

Difficulty:
Medium

Q	Topic
Q59	Docker volume drivers: NFS, EBS, GCS, Azure Disk
Q60	tmpfs for sensitive data: in-memory storage
Q61	Shared volumes between containers: correct permissions
Q62	Volume initialization: pre-populating volumes
Q63	Storage performance: volumes vs bind mounts vs overlay
Q64	Kubernetes PersistentVolumeClaim workflow

Q65

Database containers in production: volumes and backup strategy

Docker Compose

Q32. What is Docker Compose?

Difficulty: Easy

```
# docker-compose.yml: define multi-container application
version: "3.9"

services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      - DB_HOST=postgres
      - REDIS_URL=redis://redis:6379
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_started
    volumes:
      - ./config:/app/config:ro
    restart: unless-stopped
    networks:
      - backend

  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: myapp
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    command: redis-server --maxmemory 256mb --maxmemory-policy allkeys-lru

volumes:
  pgdata:

networks:
  backend:
    driver: bridge

docker compose up -d          # start all services in background
```

```
docker compose down          # stop and remove containers
docker compose down -v      # also remove volumes
docker compose logs -f app   # follow logs for app service
docker compose ps            # status of all services
docker compose exec app sh   # shell in running container
```

Q33. What are Docker Compose best practices?

Difficulty:
Medium

```
# Use environment variables for secrets (never hardcode!)
services:
  app:
    environment:
      DB_PASSWORD: ${DB_PASSWORD} # from .env file or shell env

# Separate dev vs prod compose files
# docker-compose.yml (base)
# docker-compose.override.yml (dev: bind mounts, debug ports)
# docker-compose.prod.yml (prod: resource limits, restart policies)

# docker compose -f docker-compose.yml -f docker-compose.prod.yml up

# Resource limits for production:
services:
  app:
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512M
        reservations:
          cpus: "0.25"
          memory: 128M

# Healthchecks for depends_on:
postgres:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready"]
    interval: 5s
    timeout: 5s
    retries: 5
app:
  depends_on:
    postgres:
      condition: service_healthy # wait until healthy, not just started

# Profiles for optional services:
swagger-ui:
  profiles: [dev] # only start with: docker compose --profile dev up
```

Q34. How does Docker Compose handle service dependencies?

Difficulty:
Medium

```
# depends_on: control startup order
services:
  app:
    depends_on:
```

```

- db      # wait for db container to START (not be healthy!)

# Proper healthcheck-based dependency:
app:
  depends_on:
    db:
      condition: service_healthy
    redis:
      condition: service_started # just started, no healthcheck needed

# service_healthy: waits until healthcheck returns healthy
# service_started: just started (no health check)
# service_completed_successfully: for one-shot init containers

# Problem: depends_on only controls order, not readiness
# Even with healthcheck, app may need retry logic for DB connections
# (healthcheck may pass but DB not fully ready for connections)

# Best practice: app should retry DB connections with backoff
// Go: connectWithRetry(ctx, dsn, 30, time.Second)

# Init containers pattern (docker compose):
init-db:
  image: migrate:latest
  command: ["migrate", "-path", "/migrations", "-database", "${DB_URL}", "up"]
  depends_on:
    postgres:
      condition: service_healthy

app:
  depends_on:
    init-db:
      condition: service_completed_successfully

```

Q35. ### Q69-Q75: More Compose Topics		Difficulty: Medium
Q	Topic	
Q69	Docker Compose secrets management	
Q70	Scaling services with docker compose up --scale	
Q71	Docker Compose networking: service discovery by name	
Q72	Compose file versioning and compatibility	
Q73	Docker Compose for CI/CD testing	
Q74	Override files for dev/staging/production environments	
Q75	Migrating from docker-compose to Kubernetes	

Security & Production Best Practices

Q36. What are container security best practices?

Difficulty: Hard

```
# 1. Non-root user (most important)
RUN addgroup -S app && adduser -S app -G app
USER app:app

# 2. Read-only root filesystem
docker run --read-only -v /tmp:/tmp myapp
# Or in docker-compose:
# read_only: true

# 3. Drop capabilities
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE myapp
# Default caps include many dangerous ones: drop all, add only needed

# 4. No new privileges
docker run --security-opt no-new-privileges myapp
# Prevents privilege escalation via setuid binaries

# 5. Seccomp profile (limit syscalls)
docker run --security-opt seccomp=/etc/docker/seccomp.json myapp
# Default Docker seccomp profile blocks 44+ dangerous syscalls

# 6. Minimize image (no package manager, no shell in prod)
FROM gcr.io/distroless/static:nonroot
# No shell = no shell injection attacks

# 7. Scan images for CVEs
trivy image myapp:v1

# 8. Network policies (Kubernetes)
# Restrict ingress/egress per pod

# 9. Resource limits (prevent container escape via resource exhaustion)
docker run --memory 512m --cpus 1.0 myapp

# 10. Secrets management (never ENV vars for secrets in production)
# Use: Docker secrets, Kubernetes secrets, Vault, AWS SSM
```

Q37. What is the container security attack surface?

Difficulty: Hard

Attack vectors and mitigations:

1. Vulnerable base image packages
 - Scan with Trivy, update regularly, minimal base images
2. Secrets in image layers
 - BuildKit secrets, never ENV for secrets, .dockerignore
3. Running as root
 - USER instruction, non-root base images
4. Container escape via kernel vulnerability
 - Updated kernel, gVisor (sandbox kernel), Kata Containers (VM-based)
 - Non-root reduces blast radius
5. Privilege escalation

→ --security-opt no-new-privileges, drop capabilities

6. Unrestricted syscalls

→ Seccomp profiles, AppArmor/SELinux

7. Resource abuse

→ cgroup limits (--memory, --cpus)

8. Network exposure

→ Minimal port exposure, network policies, private networks

9. Writable root filesystem

→ --read-only, separate writable volumes for /tmp, /var

10. Supply chain: compromised base images

- Pin digests (sha256), verify image provenance
- Private registries, only approved base images
- SLSA framework for build attestation

Defense in depth: apply ALL layers

Single vuln shouldn't compromise host

Q38. ### Q78-Q85: More Security Topics

Difficulty:
Medium

Q	Topic
Q78	Docker daemon security: TLS for remote socket
Q79	Image signing with Docker Content Trust and Cosign
Q80	Runtime security monitoring with Falco
Q81	Kubernetes Pod Security Admission
Q82	Secret injection: Vault Agent, AWS Secrets Manager
Q83	Container sandbox: gVisor and Kata Containers
Q84	Compliance: CIS Docker Benchmark
Q85	SAST and DAST for containerized applications

Kubernetes Basics & Go Containerization

Q39. What are Kubernetes core objects?

Difficulty:
Medium

```
# Pod: smallest unit, 1+ containers sharing network/storage
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: app
      image: myapp:v1
```

```

resources:
  requests: {cpu: "100m", memory: "128Mi"}
  limits:   {cpu: "500m", memory: "512Mi"}

# Deployment: manages replicated pods + rolling updates
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 3
  selector:
    matchLabels: {app: myapp}
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
  template:
    spec:
      containers:
        - name: app
          image: myapp:v1

# Service: stable network endpoint for pods
apiVersion: v1
kind: Service
spec:
  type: ClusterIP # or NodePort, LoadBalancer
  selector: {app: myapp}
  ports:
    - port: 80
      targetPort: 8080

# Ingress: HTTP routing to services
apiVersion: networking.k8s.io/v1
kind: Ingress
spec:
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /
            backend:
              service: {name: myapp, port: {number: 80}}
```

Q40. What are Kubernetes probes?

Difficulty:
Medium

```

containers:
- name: app
  image: myapp:v1

# Liveness: is the process alive? (restart if fails)
livenessProbe:
  httpGet:
    path: /health/live
    port: 8080
  initialDelaySeconds: 30 # wait 30s before first probe
  periodSeconds: 10      # probe every 10s
  failureThreshold: 3     # restart after 3 failures
```

```
# Readiness: is it ready to serve traffic? (remove from LB if fails)
readinessProbe:
  httpGet:
    path: /health/ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 5
  failureThreshold: 3

# Startup: for slow-starting apps (disables liveness until done)
startupProbe:
  httpGet:
    path: /health/live
    port: 8080
  failureThreshold: 30 # 30 × 10s = 5 min for startup
  periodSeconds: 10

// Go health endpoints:
http.HandleFunc("/health/live", func(w http.ResponseWriter, r *http.Request) {
  w.WriteHeader(http.StatusOK) // always 200 if process is alive
})

http.HandleFunc("/health/ready", func(w http.ResponseWriter, r *http.Request) {
  ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
  defer cancel()
  if err := db.PingContext(ctx); err != nil {
    w.WriteHeader(http.StatusServiceUnavailable)
    json.NewEncoder(w).Encode(map[string]string{"db": "unhealthy"})
    return
  }
  w.WriteHeader(http.StatusOK)
})
```

Q41. How do you configure a Go application for Kubernetes?

Difficulty:
Medium

```
# ConfigMap: non-sensitive configuration
apiVersion: v1
kind: ConfigMap
metadata: {name: myapp-config}
data:
  LOG_LEVEL: "info"
  PORT: "8080"
  config.yaml: |
    database:
      max_connections: 25

# Secret: sensitive data (base64 encoded)
apiVersion: v1
kind: Secret
metadata: {name: myapp-secrets}
type: Opaque
data:
  DB_PASSWORD: c2VjcmV0cGFzcw== # base64

# Deployment using both:
spec:
  containers:
```

```

- name: app
  image: myapp:v1
  envFrom:
  - configMapRef: {name: myapp-config}
  - secretRef: {name: myapp-secrets}
  volumeMounts:
  - name: config
    mountPath: /etc/app
volumes:
- name: config
  configMap: {name: myapp-config}

// Read from environment (Kubernetes injects):
port := os.Getenv("PORT")
if port == "" { port = "8080" }

dbPass := os.Getenv("DB_PASSWORD") // from Secret
logLevel := os.Getenv("LOG_LEVEL") // from ConfigMap

```

Q42. What is the Go binary Dockerfile for production?

Difficulty:
Medium

```

# Optimized Go production Dockerfile
FROM golang:1.22-alpine AS builder
WORKDIR /build

# Download dependencies first (stable cache layer)
COPY go.mod go.sum ./
RUN go mod download

# Copy source + build
COPY . .
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 \
    go build \
    -trimpath \
    -ldflags="-s -w -X main.version=${VERSION}" \
    -o /bin/server \
    ./cmd/server

# Minimal production image
FROM gcr.io/distroless/static:nonroot
COPY --from=builder /bin/server /server
EXPOSE 8080
ENTRYPOINT ["/server"]

# Result: ~8MB image
# No shell, no package manager
# Runs as non-root (uid 65532 = nonroot)
# CGO_ENABLED=0: fully static binary (no glibc dependency)
# -trimpath: removes build system paths from binary
# -ldflags="-s -w": strip debug symbols (smaller binary)

# Build and push:
docker build \
  --build-arg VERSION=$(git rev-parse --short HEAD) \
  -t myapp:$(git rev-parse --short HEAD) .
docker push myapp:$(git rev-parse --short HEAD)

# Verify size:

```

```
docker images myapp
# myapp: 8.2MB
```

Q43. How do you handle signals in a Go container?

Difficulty:
Medium

```
// Container MUST handle SIGTERM for graceful shutdown
// docker stop: SIGTERM → wait 10s → SIGKILL
// kubernetes: SIGTERM → wait terminationGracePeriodSeconds → SIGKILL

func main() {
    srv := &http.Server{Addr: ":8080", Handler: buildRouter()}

    // Start server in goroutine
    go func() {
        if err := srv.ListenAndServe(); !errors.Is(err, http.ErrServerClosed) {
            log.Fatal(err)
        }
    }()

    // Listen for OS signals
    quit := make(chan os.Signal, 1)
    signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
    <-quit

    log.Println("shutting down gracefully...")

    // Give 30s for in-flight requests to complete
    ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
    defer cancel()
    srv.Shutdown(ctx)

    // Close DB, message queues, etc.
    db.Close()
    log.Println("shutdown complete")
}

// Kubernetes terminationGracePeriodSeconds (default 30s):
spec:
  terminationGracePeriodSeconds: 60 # give Go app 60s to shutdown
  containers:
  - lifecycle:
      preStop:
        exec:
          command: ["/bin/sh", "-c", "sleep 5"]
        # preStop hook: small sleep to allow LB to stop routing first
```

Q44. ### Q91-Q100: Advanced Container Topics

Difficulty:
Medium

Q	Topic
Q91	Kubernetes resource requests and limits: why both matter
Q92	Horizontal Pod Autoscaler (HPA) configuration

Q93	Kubernetes ConfigMap hot-reload in Go applications
Q94	Container image optimization: multi-stage, layer order
Q95	Docker in Docker (DinD): CI/CD pipeline use case
Q96	Kubernetes namespaces for multi-tenancy
Q97	Helm charts for packaging Go applications
Q98	Kubernetes rolling deployment vs blue-green vs canary
Q99	Service mesh: Istio sidecar injection for Go services
Q100	Observability in containers: OpenTelemetry for Go

Master these 100 questions and you'll handle any Docker/Kubernetes interview at SDE2 level. Key areas: container fundamentals (namespaces, cgroups, overlay FS), Dockerfile best practices (multi-stage, distroless, non-root), Docker networking (bridge, DNS by name), Docker Compose (service dependencies, healthchecks), container security (non-root, drop caps, scan), and Go containerization (static binary, graceful shutdown, health probes). ■

Section 4 — Networking & Service Discovery (Q45–Q65)

Q45. What are Docker network types?

Difficulty:
Medium

```
# Bridge (default): isolated network, containers communicate by name
docker network create --driver bridge mynet
docker run --network mynet --name app1 myimage
docker run --network mynet --name app2 myimage
# app2 can reach app1 via http://app1:8080

# Host: container shares host network stack (no isolation)
docker run --network host nginx
# nginx listens on host port 80 directly — no port mapping needed
# Use: performance-critical, avoids NAT overhead

# None: no networking
docker run --network none myimage # fully isolated

# Overlay: multi-host networking (Docker Swarm / Kubernetes)
docker network create --driver overlay --attachable myoverlay

# Macvlan: container gets its own MAC address (appears as physical device)
docker network create --driver macvlan \
  --subnet=192.168.1.0/24 --gateway=192.168.1.1 \
  -o parent=eth0 macnet

# List networks
docker network ls
docker network inspect mynet
```

Q46. How does DNS work inside Docker?

Difficulty:
Medium

```
# User-defined networks: built-in DNS resolver
# Containers resolve each other by container name or service name

docker network create mynet
docker run -d --name postgres --network mynet postgres:16
docker run -d --name app --network mynet myapp
# app can reach postgres via: postgres:5432

# Docker daemon runs embedded DNS at 127.0.0.11
# /etc/resolv.conf inside container:
# nameserver 127.0.0.11
# options ndots:0

# Default bridge network: NO DNS (use --link, deprecated)
# Always use user-defined networks for service discovery

# Docker Compose: service name = DNS name
# services:
#   web:    → reachable as "web" from other services
#   db:     → reachable as "db"
#   redis:  → reachable as "redis"

# Aliases
docker run --network mynet --network-alias cache redis
# Now reachable as both "cache" and container name
```

Q47. What is Docker port mapping?

Difficulty: Easy

```
# -p host_port:container_port
docker run -p 8080:80 nginx      # host:8080 → container:80
docker run -p 8080:80/tcp nginx  # explicit TCP
docker run -p 5000:5000/udp myapp # UDP
docker run -p 8080:80 -p 443:443 nginx # multiple ports

# Bind to specific host interface
docker run -p 127.0.0.1:8080:80 nginx # localhost only (safer)
docker run -p 0.0.0.0:8080:80 nginx   # all interfaces (default)

# Random host port
docker run -p 80 nginx              # Docker assigns random host port
docker port container_name 80       # find assigned port

# EXPOSE in Dockerfile: documentation only (doesn't publish)
# -P flag: publish all EXPOSEd ports to random host ports
docker run -P nginx

# View port mappings
docker ps                           # shows PORTS column
docker port mycontainer
docker inspect mycontainer | grep -A 20 PortBindings
```

Q48. What is Docker Compose networking?

Difficulty:
Medium

```
# docker-compose.yml
version: "3.8"
services:
  web:
    image: myapp:latest
    ports:
      - "8080:8080"
    networks:
      - frontend
      - backend

  db:
    image: postgres:16
    networks:
      - backend
    # NOT exposed to frontend network

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    networks:
      - frontend

networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
    internal: true    # no external internet access

# web → can reach db (both on backend)
# nginx → can reach web (both on frontend)
# nginx → CANNOT reach db (not on backend)
# db → has no internet (internal: true)
```

Q49. What is multi-stage build optimization?

Difficulty: **Hard**

```
# Multi-stage: build artifacts, copy only what's needed to final image
# Result: tiny production image (no build tools)

# Stage 1: Build
FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -o server ./cmd/server

# Stage 2: Test (optional)
FROM builder AS tester
RUN go test ./...

# Stage 3: Final (minimal)
```



```
FROM gcr.io/distroless/static-debian12 AS production
COPY --from=builder /app/server /server
COPY --from=builder /app/migrations /migrations
USER nonroot:nonroot
EXPOSE 8080
ENTRYPOINT ["/server"]

# Without multi-stage: ~900MB (golang image + source + binaries)
# With multi-stage: ~10MB (just static binary)

# Build specific stage
docker build --target builder -t myapp:builder .
docker build --target production -t myapp:prod .
```

Q50. What are Docker build cache best practices?

Difficulty:
Medium

```
# Cache layers: each instruction is a layer
# Layers rebuild if instruction OR any parent layer changes

# BAD: copy everything first (invalidates cache on any file change)
FROM node:20-alpine
COPY . . # invalidates cache on every code change
RUN npm install # re-runs even if package.json unchanged

# GOOD: copy dependencies first
FROM node:20-alpine
WORKDIR /app
COPY package.json package-lock.json ./ # only invalidates if deps change
RUN npm ci # cached unless deps changed
COPY . . # code change doesn't affect npm install

# Go equivalent
FROM golang:1.22 AS builder
WORKDIR /app
COPY go.mod go.sum ./ # copy dependency files first
RUN go mod download # cached unless go.mod/go.sum changes
COPY . . # code changes don't bust go mod cache
RUN go build -o app .

# BuildKit cache mounts (persistent cache across builds)
RUN --mount=type=cache,target=/root/.cache/go-build \
    go build -o app .

# .dockerignore: exclude files that bust cache unnecessarily
# .git, *.md, tests, local configs
```

Q51. What is Docker resource limits?

Difficulty:
Medium

```
# Memory limit
docker run --memory=512m --memory-swap=512m nginx
# --memory: soft limit
# --memory-swap: memory+swap total (set equal to --memory to disable swap)

# CPU limits
docker run --cpus=1.5 nginx # 1.5 CPU cores
```

```
docker run --cpu-shares=512 nginx      # relative weight (default 1024)
docker run --cpuset-cpus="0,1" nginx  # pin to specific CPUs

# In docker-compose.yml
services:
  app:
    image: myapp
    deploy:
      resources:
        limits:
          cpus: "0.5"
          memory: 256M
        reservations:
          cpus: "0.25"
          memory: 128M

# Check resource usage
docker stats
docker stats --no-stream # one-time snapshot

# OOM killer: Docker kills container if memory exceeded
# Monitor: docker events --filter event=oom
```

Q52. What is Docker health checks?

Difficulty:
Medium

```
# HEALTHCHECK: Docker monitors container health
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:8080/health || exit 1

# Options:
# --interval: time between checks (default 30s)
# --timeout: single check timeout (default 30s)
# --start-period: grace period at startup (default 0s)
# --retries: consecutive failures to mark unhealthy (default 3)
# --start-interval: (Docker 25+) faster checks during startup

# States: starting → healthy | unhealthy

# Check from command line
docker inspect --format='{{.State.Health.Status}}' mycontainer
docker inspect --format='{{json .State.Health}}' mycontainer | jq

# docker-compose health check
services:
  db:
    image: postgres:16
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s

  app:
    depends_on:
      db:
        condition: service_healthy # wait for db healthy before starting
```

Q53. What is Docker security best practices?

Difficulty: Hard

```
# 1. Non-root user
FROM ubuntu:22.04
RUN useradd -r -u 1001 -g root appuser
USER appuser # never run as root in production

# 2. Read-only filesystem
docker run --read-only --tmpfs /tmp myimage
# OR in Compose:
# read_only: true
# tmpfs: ["/tmp"]

# 3. Drop Linux capabilities
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE nginx
# Minimal capabilities needed

# 4. No new privileges
docker run --security-opt no-new-privileges nginx

# 5. Scan images for vulnerabilities
docker scout cves myimage:latest
trivy image myimage:latest
grype myimage:latest

# 6. Use distroless or scratch base images
FROM gcr.io/distroless/static-debian12 # no shell, no package manager
FROM scratch # absolutely minimal (for static binaries)

# 7. Never store secrets in images
# BAD:
ENV DB_PASSWORD=secret123 # visible in docker history!
# GOOD: inject at runtime via environment or secrets
docker run -e DB_PASSWORD="$(vault read ...)" myimage
docker secret create db_pass secret.txt # Swarm secrets
```

Q54. What is Docker secrets management?

Difficulty: Hard

```
# Docker Secrets (Swarm mode): encrypted at rest and in transit
docker secret create db_password password.txt
docker secret create api_key - <<< "my-api-key"

# Use in service
docker service create \
  --secret db_password \
  --secret api_key \
  myimage

# In container: secrets available at /run/secrets/<name>
cat /run/secrets/db_password

# Compose with secrets
services:
  app:
    image: myapp
    secrets:
```

```

    - db_password
environment:
    DB_PASSWORD_FILE: /run/secrets/db_password

secrets:
    db_password:
        external: true # created outside compose
        # OR:
        file: ./secrets/db_password.txt # dev only

# Kubernetes alternative: kubectl create secret
# Vault: inject secrets as env at runtime (most secure)

# Runtime injection (no Swarm needed)
docker run --env-file .env.prod myimage # never commit .env.prod!
```

Q55. What is Docker Compose profiles?

Difficulty:
Medium

```

# Profiles: selectively start services
version: "3.8"
services:
    app:
        image: myapp
        # no profile: always starts

    db:
        image: postgres:16
        # no profile: always starts

    redis:
        image: redis:7
        # no profile: always starts

    pgadmin:
        image: dpage/pgadmin4
        profiles: ["debug", "tools"] # only with --profile debug/tools

    prometheus:
        image: prom/prometheus
        profiles: ["monitoring"]

    grafana:
        image: grafana/grafana
        profiles: ["monitoring"]

# Usage:
docker compose up # app + db + redis only
docker compose --profile debug up # + pgadmin
docker compose --profile monitoring up # + prometheus + grafana
docker compose --profile debug --profile monitoring up # all

# Environment variable:
COMPOSE_PROFILES=debug,monitoring docker compose up
```

Q56. What are Docker volumes vs bind mounts vs tmpfs?

Difficulty:
Medium

```
# Named Volume: Docker-managed, persists across container restarts
docker volume create pgdata
docker run -v pgdata:/var/lib/postgresql/data postgres

# Advantages: portable, Docker manages path, better performance on Linux
# Use for: database data, persistent application state

# Bind Mount: maps host path to container path
docker run -v /home/user/code:/app myimage
docker run --mount type=bind,source=$(pwd),target=/app myimage
# Use for: development (live code reload), config files, secrets

# tmpfs: in-memory, not persisted
docker run --tmpfs /tmp:size=100m,mode=1777 myimage
# Use for: sensitive data (not written to disk), temp files, performance

# Volume details
docker volume ls
docker volume inspect pgdata
docker volume rm pgdata
docker volume prune # remove unused volumes

# Backup volume
docker run --rm \
  -v pgdata:/source:ro \
  -v $(pwd):/backup \
  alpine tar czf /backup/pgdata.tar.gz -C /source .
```

Q57. What is Docker layer caching in CI/CD?

Difficulty: Hard

```
# GitHub Actions: cache Docker layers
- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3

- name: Build and push
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: myimage:latest
    cache-from: type=gha # GitHub Actions cache
    cache-to: type=gha,mode=max # max = cache all layers

# Registry cache (faster, larger)
  cache-from: type=registry,ref=myrepo/myimage:buildcache
  cache-to: type=registry,ref=myrepo/myimage:buildcache,mode=max

# Local cache
  cache-from: type=local,src=/tmp/.buildx-cache
  cache-to: type=local,dest=/tmp/.buildx-cache-new,mode=max

# BuildKit inline cache
FROM golang:1.22 AS builder
# ... build steps ...
```

```
# docker build --build-arg BUILDKIT_INLINE_CACHE=1 .
```

Q58. What is Docker BuildKit?

Difficulty: Hard

```
# BuildKit: next-gen build backend (default since Docker 23+)
# Enable manually:
DOCKER_BUILDKIT=1 docker build .
# or:
export DOCKER_BUILDKIT=1

# BuildKit features:

# 1. Parallel stages
FROM golang AS builder1
FROM python AS builder2
# BuildKit builds both simultaneously (no dependency)

# 2. Cache mounts (persistent across builds)
RUN --mount=type=cache,target=/go/pkg/mod \
    --mount=type=cache,target=/root/.cache/go-build \
    go build -o app .

# 3. Secret mounts (not stored in image layers)
RUN --mount=type=secret,id=npnrc,target=/root/.npnrc \
    npm install

docker build --secret id=npnrc,src=$HOME/.npnrc .

# 4. SSH forwarding (for private repos)
RUN --mount=type=ssh git clone git@github.com:private/repo.git

docker build --ssh default .

# 5. Inline cache
docker build --build-arg BUILDKIT_INLINE_CACHE=1 -t myimage .
```

Q59. What is Docker Compose dependency ordering?

Difficulty: Medium

```
services:
  app:
    image: myapp
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started # just started (not necessarily healthy)
    migrations:
      condition: service_completed_successfully # one-shot job completed

  db:
    image: postgres:16
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 10
```

```
redis:
  image: redis:7

migrations:
  image: myapp
  command: ["/migrate", "up"]
  depends_on:
    db:
      condition: service_healthy

# Startup order: db (healthy) + redis → migrations (completed) → app
# Shutdown order: app → migrations → db + redis (reverse)

# Important: depends_on handles ordering, NOT readiness
# App code should still handle connection retries
```

Q60. What is Docker logging drivers?

Difficulty:
Medium

```
# Logging drivers: where container logs go

# json-file (default): logs to host filesystem
docker run --log-driver json-file \
  --log-opt max-size=10m \
  --log-opt max-file=3 \
  nginx

# syslog: send to system syslog
docker run --log-driver syslog nginx

# journald: Linux systemd journal
docker run --log-driver journald nginx

# fluentd: structured logging to Fluentd
docker run --log-driver fluentd \
  --log-opt fluentd-address=localhost:24224 \
  --log-opt tag=docker.{{.Name}} \
  nginx

# awslogs: send to CloudWatch
docker run --log-driver awslogs \
  --log-opt awslogs-region=us-east-1 \
  --log-opt awslogs-group=/app/production \
  myapp

# none: disable logging
docker run --log-driver none nginx

# Configure globally in /etc/docker/daemon.json:
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" }
}
```

Q61. What is Docker image tagging strategy?

Difficulty: Easy

```
# Tagging strategy for production
```

```
# Semantic versioning
docker build -t myapp:1.2.3 .
docker build -t myapp:1.2.3 -t myapp:1.2 -t myapp:1 -t myapp:latest .

# Git SHA (immutable, traceable)
GIT_SHA=$(git rev-parse --short HEAD)
docker build -t myapp:$GIT_SHA .

# CI/CD pattern: tag with both
docker build \
  -t myapp:${GIT_SHA} \
  -t myapp:latest \
  .
docker push myapp:${GIT_SHA}
docker push myapp:latest

# In Kubernetes: always use specific tag, never :latest
# image: myapp:a1b2c3d4  ✓
# image: myapp:latest    ✗ (not reproducible, no rollback)

# Multi-arch builds
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  -t myapp:1.2.3 \
  --push .
```

Q62. What is Docker Swarm vs Kubernetes?

Difficulty:
Medium

Docker Swarm:

Pros: simple, built into Docker, easy to get started
 Cons: limited features, fewer integrations, less community
 Features: services, stacks, overlay networks, secrets, rolling updates
 Scale: hundreds of nodes max
 Use: small teams, simple orchestration needs

Kubernetes (K8s):

Pros: feature-rich, huge ecosystem, cloud-native standard
 Cons: complex, steep learning curve, resource overhead
 Features: pods, deployments, statefulsets, daemonsets, jobs,
 HPA, VPA, network policies, RBAC, CRDs, operators
 Scale: thousands of nodes
 Use: production workloads, complex microservices

Swarm commands:

```
docker swarm init
docker service create --replicas 3 myapp
docker service scale myapp=5
docker stack deploy -c docker-compose.yml mystack
```

K8s concepts vs Swarm:

```
Stack    → Namespace
Service  → Deployment + Service
Task     → Pod
Overlay  → CNI (Calico, Flannel, Cilium)
Secret   → Secret
Config   → ConfigMap
```


Q63. What is Docker Compose override files?

Difficulty:
Medium

```
# docker-compose.yml (base config)
services:
  app:
    image: myapp
    environment:
      - LOG_LEVEL=info
    ports:
      - "8080:8080"

# docker-compose.override.yml (auto-loaded in dev)
services:
  app:
    build: .           # build locally instead of pulling
    volumes:
      - ./app          # mount source for live reload
    environment:
      - LOG_LEVEL=debug
      - DEBUG=true

# docker-compose.prod.yml (explicit for prod)
services:
  app:
    deploy:
      replicas: 3
      restart_policy:
        condition: on-failure

# Commands:
docker compose up                # base + override (dev)
docker compose -f docker-compose.yml \
    -f docker-compose.prod.yml up  # base + prod
docker compose -f docker-compose.yml up  # base only
```

Q64. What is Docker init process and PID 1?

Difficulty: Hard

```
# PID 1 problem: process must handle signals and reap zombies
# Most apps don't handle SIGTERM properly as PID 1

# BAD: node/python/etc. as PID 1 (doesn't handle SIGTERM properly)
CMD ["node", "server.js"]

# GOOD: use tini as init process
FROM node:20-alpine
RUN apk add --no-cache tini
ENTRYPOINT ["/sbin/tini", "--"]
CMD ["node", "server.js"]

# Docker 1.13+: --init flag adds tini automatically
docker run --init myimage

# tini responsibilities:
# - Forwards signals to child process
# - Reaps zombie processes (waitpid)
```

```
# - Exits with child's exit code

# ENTRYPOINT vs CMD:
# ENTRYPOINT: fixed executable
# CMD: default arguments (overridable)
# Both: ["exec", "form"] vs "shell form"
# Shell form: /bin/sh -c "command" (extra process, no signal forwarding)
# Exec form: ["command", "arg"] (direct execution, correct PID 1)

# Correct:
ENTRYPOINT ["/sbin/tini", "--"]
CMD ["/app/server"]
```

Q65. What is Docker Compose environment variable handling?

Difficulty: Easy

```
# Methods to inject environment variables

# 1. Hardcoded (don't use for secrets)
environment:
  APP_ENV: production
  PORT: "8080"

# 2. From host environment (no = means take from host)
environment:
  - DB_PASSWORD      # taken from host: export DB_PASSWORD=secret
  - API_KEY

# 3. From .env file (auto-loaded from same directory)
# .env:
# DB_PASSWORD=secret
# API_KEY=abc123
environment:
  - DB_PASSWORD
  - API_KEY

# 4. env_file directive
env_file:
  - .env
  - .env.local      # overrides .env

# 5. Variable substitution in docker-compose.yml
services:
  app:
    image: myapp:${APP_VERSION:-latest} # default: latest
    ports:
      - "${HOST_PORT:-8080}:8080"

# .env variable expansion
# DATABASE_URL=postgres://${DB_USER}:${DB_PASS}@db:5432/${DB_NAME}
```

Section 5 — Production, CI/CD & Advanced Topics (Q66–Q110)

Q66. What is Docker image scanning?

Difficulty:
Medium

```
# Trivy (most popular)
trivy image myapp:latest
trivy image --severity HIGH,CRITICAL myapp:latest
trivy image --format json -o results.json myapp:latest

# Docker Scout (built-in)
docker scout cves myapp:latest
docker scout recommendations myapp:latest
docker scout compare myapp:latest myapp:previous

# Snyk
snyk container test myapp:latest
snyk container monitor myapp:latest

# In CI/CD (fail on critical)
trivy image --exit-code 1 --severity CRITICAL myapp:latest

# Scan Dockerfile for issues
docker run --rm -i hadolint/hadolint < Dockerfile

# Common vulnerabilities:
# - Base image CVEs (update base image regularly)
# - Outdated packages (RUN apt-get upgrade)
# - Running as root
# - Hardcoded secrets
# - Unnecessary packages (increase attack surface)
```

Q67. What is Docker Registry?

Difficulty: Easy

```
# Docker Hub (default public registry)
docker pull nginx:latest
docker push myorg/myapp:1.0.0

# Private registry (self-hosted)
docker run -d -p 5000:5000 --name registry \
  -v /mnt/registry:/var/lib/registry \
  registry:2

docker tag myapp localhost:5000/myapp:1.0.0
docker push localhost:5000/myapp:1.0.0
docker pull localhost:5000/myapp:1.0.0

# Cloud registries
# AWS ECR: 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp
# GCR: gcr.io/project-id/myapp
# GHA: ghcr.io/org/myapp
# Azure ACR: myregistry.azurecr.io/myapp

# Login
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin \
  123456789.dkr.ecr.us-east-1.amazonaws.com

# Configure daemon for insecure registry
```

```
# /etc/docker/daemon.json:
{ "insecure-registries": ["localhost:5000"] }
```

Q68. What are Docker networking problems and debugging?

Difficulty: Hard

```
# Debug container networking
docker exec mycontainer ip addr
docker exec mycontainer ip route
docker exec mycontainer cat /etc/resolv.conf
docker exec mycontainer ping db          # test DNS
docker exec mycontainer curl http://db:5432 # test connectivity

# Network inspection
docker network inspect mynet
docker network inspect bridge

# Port conflicts
ss -tlnp | grep :8080    # check if port in use on host
netstat -tlnp | grep :8080

# Container can't reach external internet
# Check: iptables rules, docker network, host firewall
iptables -L -n -t nat    # view NAT rules
sudo sysctl net.ipv4.ip_forward # should be 1

# Container DNS not resolving
docker exec mycontainer nslookup google.com
# If failing: check /etc/resolv.conf, try --dns flag
docker run --dns 8.8.8.8 myimage

# Common fixes
docker network prune      # remove unused networks
docker restart mycontainer # restart to re-attach network
systemctl restart docker  # reset all networking (last resort)
```

Q69. What is Docker Compose in production?

Difficulty: Hard

```
# Production docker-compose considerations

version: "3.8"
services:
  app:
    image: myapp:${APP_VERSION} # always pinned version
    restart: unless-stopped
    deploy:
      replicas: 2
      update_config:
        parallelism: 1
        delay: 10s
        failure_action: rollback
      rollback_config:
        parallelism: 1
      restart_policy:
        condition: on-failure
        max_attempts: 3
    healthcheck:
```

```

    test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 40s
environment:
  - DB_URL=${DB_URL}          # from environment, not hardcoded
logging:
  driver: "json-file"
  options:
    max-size: "50m"
    max-file: "5"
resources:
  limits:
    cpus: "0.5"
    memory: 512M

```

Q70. What is Docker exec vs docker run vs docker attach?

Difficulty: Easy

```

# docker run: create and start a NEW container
docker run -it ubuntu bash          # interactive new container
docker run -d myapp                 # detached (background)
docker run --rm myapp               # auto-remove when done

# docker exec: run command in RUNNING container
docker exec mycontainer ls /app
docker exec -it mycontainer bash     # interactive shell
docker exec -it mycontainer sh -c "ps aux | grep app"
docker exec -u root mycontainer apt-get install -y curl # as root

# docker attach: attach to PID 1 of RUNNING container
docker attach mycontainer           # attach to main process stdin/stdout
# Ctrl+P, Ctrl+Q to detach without stopping
# Ctrl+C to send SIGINT (may stop container)

# docker logs: view container stdout/stderr
docker logs mycontainer
docker logs -f mycontainer          # follow (like tail -f)
docker logs --tail 100 mycontainer # last 100 lines
docker logs --since 1h mycontainer # last 1 hour

```

Q71. What is Dockerfile CMD vs ENTRYPOINT?

Difficulty: Medium

```

# CMD: default command (overridable at docker run)
CMD ["nginx", "-g", "daemon off;"]

# docker run myimage          → runs nginx
# docker run myimage bash     → runs bash instead

# ENTRYPOINT: fixed executable (not overridable without --entrypoint)
ENTRYPOINT ["nginx"]
CMD ["-g", "daemon off;"]

# docker run myimage          → nginx -g daemon off;
# docker run myimage -t       → nginx -t (tests config)
# docker run --entrypoint bash myimage → bash (override entrypoint)

# Shell vs exec form

```

```
# Shell form: /bin/sh -c "command" (signal issues, extra process)
CMD nginx -g "daemon off;" # shell form (BAD for PID 1)
CMD ["nginx", "-g", "daemon off;"] # exec form (GOOD)

# Pattern: entrypoint script for setup + exec
ENTRYPOINT ["/docker-entrypoint.sh"]
CMD ["server"]

# docker-entrypoint.sh
#!/bin/sh
set -e
if [ "$1" = "server" ]; then
    run_migrations
    exec /app/server
elif [ "$1" = "worker" ]; then
    exec /app/worker
fi
exec "$@" # pass through unknown commands
```

Q72. What is Docker image layer inspection?

Difficulty:
Medium

```
# View image layers
docker history myimage:latest
docker history --no-trunc myimage:latest # full commands

# Layer analysis with dive (tool)
docker run --rm -it -v /var/run/docker.sock:/var/run/docker.sock \
    wagooodman/dive myimage:latest

# Inspect image metadata
docker inspect myimage:latest
docker inspect --format '{{.Size}}' myimage:latest # total size in bytes
docker inspect --format '{{len .RootFS.Layers}}' myimage # layer count

# Image size breakdown
docker image ls --format "{{.Repository}}:{{.Tag}}\t{{.Size}}"

# What adds to image size:
# - Every RUN command creates a layer
# - Deleted files in new layer still exist in previous layer!
# Solution: clean up in SAME RUN instruction

# BAD: delete in separate RUN (both layers kept)
RUN apt-get update
RUN apt-get install -y build-essential
RUN rm -rf /var/lib/apt/lists/* # too late! previous layers already have it

# GOOD: single RUN, cleanup in same layer
RUN apt-get update && \
    apt-get install -y build-essential && \
    rm -rf /var/lib/apt/lists/*
```

Q73. What is Docker context?

Difficulty:
Medium

```
# Docker context: manage connections to multiple Docker hosts/environments
```

```

docker context ls
docker context create prod \
  --docker "host=ssh://user@prod.example.com"
docker context use prod          # switch to prod
docker ps                       # shows prod containers
docker context use default      # back to local

# Docker context with TLS
docker context create remote \
  --docker "host=tcp://remote:2376,ca=/certs/ca.pem,cert=/certs/cert.pem,key=/certs/key.pem"

# In CI: DOCKER_HOST environment variable
DOCKER_HOST=ssh://deploy@prod.example.com docker ps

# With Docker Compose
docker context use prod
docker compose -f docker-compose.prod.yml up -d
# Runs on prod host!

# Prune remote
docker --context prod system prune -f

# Export/import context
docker context export prod > prod.dockercontext
docker context import prod prod.dockercontext

```

Q74. What is Docker containerd and low-level architecture?

Difficulty: Hard

Docker architecture (modern):

```

CLI → Docker daemon (dockerd)
      ↓
    containerd (container runtime)
      ↓
    runc (OCI runtime, actually creates containers)
      ↓
    Linux kernel (namespaces, cgroups)

```

Components:

```

dockerd:    Docker daemon, manages high-level objects (images, networks, volumes)
containerd: Low-level container lifecycle (create, start, stop, delete)
            Also used by Kubernetes directly (bypasses Docker daemon)
runc:       OCI runtime - actual container creation using kernel features
containerd-shim: keeps container running even if containerd restarts

```

OCI (Open Container Initiative):

```

Image spec: how images are structured
Runtime spec: how containers are executed
Distribution spec: how images are distributed (registry API)

```

Why Kubernetes removed Docker:

```

K8s only needs containerd (not full Docker daemon)
Docker adds overhead not needed in K8s
containerd is leaner, K8s-native
cri-o: alternative runtime used in OpenShift

```

Q75. What is Docker network debugging with tcpdump?

Difficulty: Hard

```
# Debug container network traffic

# Method 1: nsenter (enter network namespace)
PID=$(docker inspect --format '{{.State.Pid}}' mycontainer)
nsenter -t $PID -n tcpdump -i eth0 port 5432

# Method 2: run tcpdump container in target's network namespace
docker run --rm --net container:mycontainer \
    nicolaka/netshoot tcpdump -i eth0 -n

# Method 3: inspect with netshoot (debug container)
docker run --rm --net container:db \
    nicolaka/netshoot \
    nmap -p 5432 localhost

# Common issues and diagnosis:
# 1. Connection refused: service not listening
docker exec db netstat -tlnp | grep 5432

# 2. Connection timeout: firewall/network policy
iptables -L DOCKER-USER -n

# 3. Name resolution failing
docker exec app nslookup db
docker exec app cat /etc/hosts
docker exec app cat /etc/resolv.conf

# 4. Container can't reach internet (check bridge IP forwarding)
sudo sysctl -w net.ipv4.ip_forward=1
```

Q76. What is Docker garbage collection / prune?

Difficulty: Easy

```
# Prune everything unused
docker system prune          # stopped containers, unused images, networks
docker system prune -a       # + unused images (not just dangling)
docker system prune -a --volumes # + volumes (dangerous!)
docker system prune --force    # no confirmation prompt

# Specific prune
docker container prune        # stopped containers
docker image prune            # dangling images (no tag)
docker image prune -a         # all unused images
docker volume prune           # unused volumes
docker network prune          # unused networks
docker builder prune          # build cache

# Space usage
docker system df              # disk usage summary
docker system df -v           # verbose (per image/container/volume)

# Automated cleanup (cron job)
# 0 2 * * * /usr/bin/docker system prune -af --volumes 2>&1

# In production: be careful with volume prune!
# Named volumes used by stopped containers will be deleted
docker volume prune --filter "label!=keep"
```


Q77. What is Docker multi-arch builds?

Difficulty: Hard

```
# Build for multiple CPU architectures (AMD64 + ARM64)
# Required for: M1/M2 Macs, Raspberry Pi, AWS Graviton

# Setup buildx
docker buildx create --name multiarch --driver docker-container --use
docker buildx inspect --bootstrap

# Build and push multi-arch image
docker buildx build \
  --platform linux/amd64,linux/arm64,linux/arm/v7 \
  --tag myapp:1.0.0 \
  --push \
  .

# Verify manifest
docker manifest inspect myapp:1.0.0

# In Dockerfile: use TARGETPLATFORM for platform-specific steps
FROM --platform=$BUILDPLATFORM golang:1.22 AS builder
ARG TARGETOS TARGETARCH
RUN GOOS=$TARGETOS GOARCH=$TARGETARCH go build -o app .

FROM --platform=$TARGETPLATFORM alpine:3.19
COPY --from=builder /app /app
ENTRYPOINT ["/app"]

# Emulation: QEMU for cross-compilation
docker run --privileged --rm tonistiigi/binfmt --install all
```

Q78. What is Docker Swarm services and stacks?

Difficulty: Hard

```
# Initialize Swarm
docker swarm init --advertise-addr 192.168.1.10
# Join token shown → run on worker nodes:
docker swarm join --token SWMTKN-... 192.168.1.10:2377

# Deploy service
docker service create \
  --name webapp \
  --replicas 3 \
  --publish published=80,target=8080 \
  --update-delay 10s \
  --update-parallelism 1 \
  myapp:1.0.0

# Scale
docker service scale webapp=5

# Rolling update
docker service update \
  --image myapp:2.0.0 \
  --update-parallelism 1 \
  --update-delay 30s \
  webapp
```

```
# Rollback
docker service rollback webapp

# Deploy stack (compose file)
docker stack deploy -c docker-compose.prod.yml mystack
docker stack ls
docker stack services mystack
docker stack rm mystack

# Node management
docker node ls
docker node update --availability drain worker1 # drain before maintenance
```

Q79. What is Docker storage drivers?

Difficulty: Hard

Storage drivers: manage image layers and container writeable layer

overlay2 (default, recommended):

- Uses Linux OverlayFS
- Lower layer: read-only image layers
- Upper layer: writable container layer
- Best performance on most Linux systems

devicemapper:

- Used on older kernels
- Each container gets its own thin-provisioned device
- Slower than overlay2
- Legacy: avoid if possible

btrfs:

- Requires btrfs filesystem
- Efficient snapshots
- Less common

zfs:

- Requires ZFS filesystem
- Strong data integrity
- Less common

Check storage driver:

```
docker info | grep "Storage Driver"
```

Performance considerations:

- overlay2: best for most workloads
- Avoid frequent small writes to container layer
- Use volumes for write-heavy data (bypasses storage driver)
- Databases should ALWAYS use volumes

Change storage driver:

```
/etc/docker/daemon.json:
{ "storage-driver": "overlay2" }
```

Q80. What are Docker Compose wait strategies?

Difficulty: Medium

```
# Problem: docker-compose up starts all at once
# App starts before DB is ready → connection refused
```

```
# Solution 1: wait-for-it.sh script
services:
  app:
    command: ["/wait-for-it.sh", "db:5432", "--", "./app"]
    # Polls TCP port until open

# Solution 2: dockerize tool
services:
  app:
    command: >
      dockerize -wait tcp://db:5432
        -wait tcp://redis:6379
        -timeout 60s
        ./app

# Solution 3: healthcheck + depends_on (best)
services:
  db:
    image: postgres
    healthcheck:
      test: ["CMD-SHELL", "pg_isready"]
      interval: 5s; retries: 10
  app:
    depends_on:
      db:
        condition: service_healthy

# Solution 4: application-level retry (most resilient)
func connectDB() *sql.DB {
    var db *sql.DB
    var err error
    for i := 0; i < 30; i++ {
        db, err = sql.Open("postgres", dsn)
        if err == nil {
            if err = db.Ping(); err == nil { return db }
        }
        log.Printf("DB not ready, retrying in 2s... (%d/30)", i+1)
        time.Sleep(2 * time.Second)
    }
    log.Fatal("DB never became ready")
    return nil
}
```

Q81. ### Q81–Q110: Advanced Docker Topics

Difficulty:
Medium

Q82. What is Docker COPY vs ADD?

Difficulty:
Medium

```
# COPY: simple copy from build context to image (preferred)
COPY src/ /app/src/
COPY --chown=appuser:appuser config.yaml /app/
COPY --chmod=755 entrypoint.sh /entrypoint.sh

# ADD: like COPY but also:
#   - Auto-extracts tar archives
```

```
# - Can fetch from URLs (AVOID: no caching, security risk)
ADD archive.tar.gz /app/      # extracts archive
ADD https://example.com/file /tmp/ # BAD: no cache, security risk

# Best practice: always use COPY unless you need ADD's tar extraction
# For URL downloads, use RUN curl/wget (better layer caching control)
RUN curl -fsSL https://example.com/file -o /tmp/file && \
    verify_checksum /tmp/file && \
    install /tmp/file /usr/local/bin/ && \
    rm /tmp/file
```

Q83. What is .dockerignore?

Difficulty:
Medium

```
# .dockerignore: exclude files from build context (like .gitignore)
# Reduces build context size → faster builds
# Prevents sensitive files from being copied
```

```
.git
.github
*.md
*.test.go
*_test.go
tests/
docs/
.env
.env.*
secrets/
*.pem
*.key
node_modules/
vendor/
.DS_Store
Thumbs.db
coverage.out
*.prof
```

```
# Pattern syntax (same as .gitignore)
**/*.log      # any .log file in any subdirectory
!important.log # except this one
```

Q84. What is Dockerfile USER instruction?

Difficulty:
Medium

```
# Run as non-root (security best practice)
FROM ubuntu:22.04

# Create non-root user
RUN groupadd -r appgroup && \
    useradd -r -g appgroup -s /bin/false -d /app appuser

# Create app directory with correct ownership
RUN mkdir -p /app && chown appuser:appgroup /app
WORKDIR /app

# Install dependencies as root
RUN apt-get update && apt-get install -y ca-certificates && rm -rf /var/lib/apt/lists/*
```

```
# Switch to non-root
COPY --chown=appuser:appgroup . .
USER appuser

# Verify in CI:
# docker run myimage id → should NOT show uid=0(root)

# Distroless: nonroot user predefined
FROM gcr.io/distroless/static-debian12:nonroot
COPY app /app
ENTRYPOINT ["/app"]
```

Q85. What is Docker container lifecycle states?

Difficulty:
Medium

Container states:

```
created    → docker create (not running)
running    → docker start / docker run
paused     → docker pause (SIGSTOP)
restarting → restart policy triggered
exited     → process stopped (exit code stored)
dead       → removal failed (rare, cleanup needed)
removing   → docker rm in progress
```

State transitions:

```
docker run    → created → running
docker stop   → running → exited (SIGTERM, wait, SIGKILL)
docker kill   → running → exited (immediate SIGKILL)
docker pause  → running → paused
docker unpause → paused → running
docker restart → running → exited → running
```

Inspect state:

```
docker inspect --format '{{.State.Status}}' container
docker inspect --format '{{.State.ExitCode}}' container
docker wait container # blocks until container exits, returns exit code
```

Q86. What is Docker daemon configuration?

Difficulty:
Medium

```
// /etc/docker/daemon.json
{
  "data-root": "/data/docker",          // storage location
  "storage-driver": "overlay2",
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "100m",
    "max-file": "5"
  },
  "default-ulimits": {
    "nofile": { "Name": "nofile", "Hard": 65536, "Soft": 65536 }
  },
  "live-restore": true,                  // containers survive daemon restart
  "max-concurrent-downloads": 10,
  "max-concurrent-uploads": 5,
  "registry-mirrors": ["https://mirror.example.com"],
  "insecure-registries": ["localhost:5000"],
```

```

    "dns": ["8.8.8.8", "8.8.4.4"],
    "experimental": false,
    "metrics-addr": "0.0.0.0:9323", // Prometheus metrics
    "userns-remap": "default"       // user namespace isolation
}

```

Q87. What is Docker networking with Compose for microservices?

Difficulty:
Medium

```

# Microservices: isolate networks per concern
version: "3.8"
services:
  api-gateway:
    networks: [public, internal]

  user-service:
    networks: [internal, user-db-net]

  order-service:
    networks: [internal, order-db-net]

  user-db:
    networks: [user-db-net]    # only accessible by user-service

  order-db:
    networks: [order-db-net]   # only accessible by order-service

networks:
  public:      # internet-facing
  internal:    # service-to-service
    internal: true # no external access
  user-db-net:
    internal: true
  order-db-net:
    internal: true

```

Q88. What is Docker ONBUILD instruction?

Difficulty:
Medium

```

# ONBUILD: trigger instruction when image is used as base
# Deferred: runs when child image is built, not current build

# Base image (node-app-base)
FROM node:20-alpine
RUN mkdir -p /app
WORKDIR /app
ONBUILD COPY package*.json ./      # runs when derived image builds
ONBUILD RUN npm ci                  # runs when derived image builds
ONBUILD COPY . .                    # runs when derived image builds

# Child image (just 2 lines!)
FROM myorg/node-app-base:latest
EXPOSE 3000
CMD ["node", "server.js"]

# docker build -f Dockerfile.child → triggers ONBUILD steps from parent
# Use case: standardize base images across many similar apps

```

Less popular now with multi-stage builds

Q89. What is Docker content trust and image signing?

Difficulty:
Medium

```
# Docker Content Trust (DCT): sign and verify images
export DOCKER_CONTENT_TRUST=1
docker pull nginx:latest    # verifies signature
docker push myapp:1.0.0     # signs image

# Generate keys
docker trust key generate mykey

# Sign image
docker trust sign myapp:1.0.0

# View signatures
docker trust inspect myapp:1.0.0

# Cosign (modern, Sigstore standard)
cosign sign myapp:1.0.0
cosign verify myapp:1.0.0

# SBOM (Software Bill of Materials)
docker sbom myapp:1.0.0
syft myapp:1.0.0 -o spdx-json > sbom.json

# In K8s: admission controller to enforce signed images
# Kyverno, OPA Gatekeeper, Connaisseur
```

Q90. What is Docker registry authentication?

Difficulty:
Medium

```
# Login to registries
docker login                                # Docker Hub
docker login registry.example.com          # private registry
docker login -u USERNAME -p PASSWORD registry.example.com

# Credentials stored in ~/.docker/config.json
# Plaintext base64 by default!

# Use credential helper (more secure)
# macOS: docker-credential-osxkeychain (built-in)
# Linux: docker-credential-pass (uses gpg-encrypted store)
# AWS: docker-credential-ecr-login

# ~/.docker/config.json
{
  "credHelpers": {
    "123456789.dkr.ecr.us-east-1.amazonaws.com": "ecr-login",
    "gcr.io": "gcloud"
  },
  "credsStore": "osxkeychain"
}

# In CI: use service account / OIDC tokens
# GitHub Actions:
```

```
- uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${github.actor}
    password: ${secrets.GITHUB_TOKEN}
```

Q91. What is Docker Compose watch (file sync)?

Difficulty:
Medium

```
# Docker Compose Watch (Compose 2.22+): hot reload in dev
services:
  app:
    build: .
    develop:
      watch:
        - action: sync          # sync files without rebuild
          path: ./src
          target: /app/src

        - action: rebuild      # rebuild image when these change
          path: go.mod

        - action: rebuild
          path: go.sum

        - action: rebuild
          path: Dockerfile

        - action: sync+restart  # sync and restart container
          path: ./config
          target: /app/config

# Run
docker compose watch
# Watches for changes, applies action automatically

# vs bind mount:
# bind mount: immediate sync but bypasses Docker layer caching
# watch: more controlled, rebuild only when needed
```

Q92. What is Docker networking for database connections?

Difficulty:
Medium

```
services:
  app:
    environment:
      # Connect to db using service name (Docker DNS)
      DATABASE_URL: "postgres://user:pass@db:5432/mydb"
      REDIS_URL: "redis://redis:6379"
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: user
```



```

    POSTGRES_PASSWORD: pass
    POSTGRES_DB: mydb
volumes:
  - pgdata:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U user -d mydb"]
  interval: 5s
  retries: 10

```

```

redis:
  image: redis:7-alpine
  volumes:
    - redisdata:/data

```

```

volumes:
  pgdata:
  redisdata:

```

Q93. ### Q92–Q110: Final Docker Questions

Difficulty:
Medium

Q	Topic
Q92	Docker cgroup v2 and resource control
Q93	Docker rootless mode
Q94	Docker in Docker (DinD) for CI
Q95	Docker Compose extends and anchors (YAML)
Q96	Distroless vs scratch vs alpine base images
Q97	Docker performance tuning (I/O, memory)
Q98	Docker checkpoint and restore (CRIU)
Q99	OCI image format internals
Q100	Docker Compose for integration testing
Q101	Kubernetes pod spec vs Docker run mapping
Q102	Docker layer squashing
Q103	Container escape prevention
Q104	Docker event stream monitoring
Q105	Docker Compose scale vs deploy replicas
Q106	Build args vs environment variables
Q107	Docker registry garbage collection
Q108	Docker socket security risks
Q109	Dockerfile best practices summary
Q110	Production Docker checklist

Q94. What is Docker in Docker (DinD)?

Difficulty: Hard

```

# DinD: run Docker daemon inside a container (used in CI)
# Use case: CI job needs to build Docker images

```

```
# Method 1: DinD (privileged, risky)
services:
  ci-runner:
    image: docker:24-dind
    privileged: true
    environment:
      DOCKER_TLS_CERTDIR: /certs
    volumes:
      - docker-certs:/certs

# Method 2: Docker socket bind mount (share host Docker)
services:
  ci-runner:
    image: docker:24-cli
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    # Risky: full host Docker access

# Method 3: Kaniko (build without Docker daemon)
# Used in K8s CI (no privileged needed)
- name: Build image
  image: gcr.io/kaniko-project/executor:latest
  args:
    - "--dockerfile=Dockerfile"
    - "--destination=myrepo/myapp:latest"

# Method 4: Buildah (rootless image building)
buildah bud -t myapp:latest .
buildah push myapp:latest
```

Q95. What is Docker Compose YAML anchors?

Difficulty:
Medium

```
# YAML anchors: reuse config blocks
x-common: &common
  restart: unless-stopped
  logging:
    driver: json-file
    options:
      max-size: 10m
      max-file: "3"
  networks:
    - app-net

x-healthcheck: &healthcheck
  interval: 30s
  timeout: 10s
  retries: 3
  start_period: 30s

services:
  web:
    <<: *common          # merge anchor
    image: myapp:latest
    healthcheck:
      <<: *healthcheck
      test: ["CMD", "curl", "-f", "http://localhost/health"]
```

```
worker:
  <<: *common
  image: myapp:latest
  command: worker
  healthcheck:
    <<: *healthcheck
    test: ["CMD", "./healthcheck-worker"]
```

Q96. What are distroless vs scratch vs alpine?

Difficulty:
Medium

scratch:
 Literally empty image (0 bytes)
 Only for fully static binaries (CGO_ENABLED=0)
 FROM scratch
 COPY app /app
 Size: binary size only (~5-10MB for Go app)
 Cons: no shell, no debugging tools

distroless (Google):
 Minimal OS (libc, ca-certs, tzdata only)
 No shell, no package manager
 gcr.io/distroless/static-debian12 (for static binaries)
 gcr.io/distroless/base-debian12 (for dynamic binaries)
 gcr.io/distroless/java21 (for Java)
 Size: ~2-5MB + binary
 Debug variant: :debug tag includes busybox shell

alpine:
 Full musl-libc Linux (~5MB base)
 Has shell, apk package manager
 FROM alpine:3.19
 RUN apk add --no-cache ca-certificates
 Size: ~5MB + binary + packages
 Good for: apps needing runtime packages, debugging

Recommendation:
 Production Go: distroless/static or scratch
 Production Java/Python: distroless/base
 Development/debugging: alpine

Q97. What is Docker build optimization checklist?

Difficulty:
Medium

Checklist for optimal Docker images:

1. ■ Use specific base image tags (not :latest)
 FROM golang:1.22.3-alpine3.19

2. ■ Multi-stage builds
 FROM golang:1.22 AS builder
 FROM gcr.io/distroless/static AS final

3. ■ Minimize layers (chain RUN commands)
 RUN apt-get update && \
 apt-get install -y --no-install-recommends ca-certificates && \
 rm -rf /var/lib/apt/lists/*

```
# 4. ■ Copy dependency files before source code
COPY go.mod go.sum ./
RUN go mod download
COPY . .

# 5. ■ Non-root user
USER nonroot:nonroot

# 6. ■ .dockerignore to exclude unnecessary files

# 7. ■ Use COPY not ADD

# 8. ■ Exec form for CMD/ENTRYPOINT
CMD ["/app/server"]

# 9. ■ BuildKit cache mounts for package managers
RUN --mount=type=cache,target=/go/pkg/mod go build

# 10. ■ Scan for vulnerabilities
# trivy image myapp:latest

# Result: small (10-20MB), secure, fast-building images
```

Q98. What is Docker system monitoring?

Difficulty:
Medium

```
# Container resource monitoring
docker stats                                # live stats all containers
docker stats container1 container2          # specific containers
docker stats --no-stream                    # one-time snapshot
docker stats --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"

# Events
docker events                               # live events
docker events --since 1h                    # last hour
docker events --filter event=die            # only container deaths
docker events --filter container=myapp

# Prometheus metrics from Docker
# daemon.json: "metrics-addr": "0.0.0.0:9323"
curl http://localhost:9323/metrics

# cAdvisor: container metrics for Prometheus
docker run -d \
  --name cadvisor \
  --volume /var/run/docker.sock:/var/run/docker.sock:ro \
  --volume /sys:/sys:ro \
  --volume /var/lib/docker:/var/lib/docker:ro \
  -p 8080:8080 \
  gcr.io/cadvisor/cadvisor:latest

# Key metrics to monitor:
# container_cpu_usage_seconds_total
# container_memory_usage_bytes
# container_network_receive_bytes_total
# container_fs_usage_bytes
```

Q99. What is Docker socket and security risks?

Difficulty:
Medium

`/var/run/docker.sock`: Unix socket for Docker daemon

Binding `docker.sock` into container = root access to host!

- Container can: create privileged containers
- Mount host filesystem: `docker run -v /:/host busybox`
- Escape container: `docker run --privileged --pid=host myimage nsenter -t 1 -m -u -i -n sh`

Mitigations:

1. Never bind `docker.sock` in production
2. Use rootless Docker (daemon runs as non-root user)
3. Use socket proxy (Tecnativa/docker-socket-proxy) to restrict API
4. Use Kaniko/Buildah for CI image builds (no socket needed)
5. Enable Docker user namespace remapping

Docker socket proxy:

Allows only specific Docker API endpoints

```
docker run -d \
  -e CONTAINERS=1 \    # allow read containers
  -e IMAGES=1 \        # allow read images
  -v /var/run/docker.sock:/var/run/docker.sock \
  tecnativa/docker-socket-proxy:latest
# Then mount the proxy socket, not the real one
```

Q100. What is the Docker production readiness checklist?

Difficulty:
Medium

Before deploying containers to production:

Image:

- Multi-stage build (minimal final image)
- Non-root user
- Specific tag pinned (not `:latest`)
- Vulnerability scan passing (no CRITICAL CVEs)
- Signed image (cosign/DCT)
- `.dockerignore` configured

Runtime:

- Memory limits set
- CPU limits set
- Health check configured
- Restart policy: `unless-stopped` or `on-failure`
- Read-only filesystem where possible
- No privileged mode
- Capabilities dropped (`--cap-drop ALL`)

Networking:

- No unnecessary ports exposed
- Internal services on isolated networks
- TLS for external traffic

Logging:

- Structured logging (JSON)
- Log rotation configured (`max-size`, `max-file`)
- Centralized log aggregation (ELK, CloudWatch)

Observability:

- Prometheus metrics exposed
- Distributed tracing (OpenTelemetry)
- Container metrics via cAdvisor

Secrets:

- No secrets in image (docker history is readable)
- Secrets injected at runtime (env, Docker secrets, Vault)
- .env files not committed to git

Q101. What is Docker compose for local development?**Difficulty:**
Medium

```
# Ideal local dev docker-compose.yml
version: "3.8"
services:
  app:
    build:
      context: .
      target: dev                # build to dev stage only
    volumes:
      - ../app                  # live code reload
      - /app/vendor              # don't mount vendor (use container's)
    ports:
      - "8080:8080"
      - "2345:2345"              # delve debugger port
    environment:
      - APP_ENV=development
      - DEBUG=true
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16-alpine
    ports:
      - "5432:5432"              # expose for local DB tools (TablePlus, etc.)
    environment:
      POSTGRES_PASSWORD: devpass
      POSTGRES_DB: myapp_dev
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./migrations:/docker-entrypoint-initdb.d # auto-run on first start
    healthcheck:
      test: ["CMD-SHELL", "pg_isready"]
      interval: 5s; retries: 10

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

volumes:
  pgdata:
```

Q102. What is Docker Compose for integration testing?

Difficulty:
Medium

```
// testcontainers-go: spin up real Docker containers in tests
import "github.com/testcontainers/testcontainers-go/modules/postgres"

func TestWithRealDB(t *testing.T) {
    ctx := context.Background()

    pgContainer, err := postgres.RunContainer(ctx,
        testcontainers.WithImage("postgres:16-alpine"),
        postgres.WithDatabase("testdb"),
        postgres.WithUsername("test"),
        postgres.WithPassword("test"),
        testcontainers.WithWaitStrategy(
            wait.ForLog("database system is ready to accept connections"),
        ),
    )
    require.NoError(t, err)
    defer pgContainer.Terminate(ctx)

    connStr, _ := pgContainer.ConnectionString(ctx, "sslmode=disable")

    db, err := pgxpool.New(ctx, connStr)
    require.NoError(t, err)

    // Real integration test with actual Postgres!
    repo := NewUserRepository(db)
    user, err := repo.Create(ctx, &User{Name: "Alice"})
    require.NoError(t, err)
    assert.NotZero(t, user.ID)
}
```

Q103. What is Docker manifest and multi-arch?

Difficulty:
Medium

```
# Docker manifest: metadata about image (list of platform-specific images)

# View manifest list
docker manifest inspect nginx:latest
# Shows: manifests for linux/amd64, linux/arm64, linux/arm/v7, etc.

# Create manifest list manually
docker manifest create myapp:1.0.0 \
    --amend myapp:1.0.0-amd64 \
    --amend myapp:1.0.0-arm64

docker manifest annotate myapp:1.0.0 myapp:1.0.0-arm64 \
    --arch arm64 --os linux

docker manifest push myapp:1.0.0

# Docker pull: automatically selects correct platform
# On ARM64 Mac: docker pull nginx → pulls linux/arm64 image
# On AMD64 Linux: docker pull nginx → pulls linux/amd64 image

# Force specific platform
```

```
docker pull --platform linux/amd64 nginx:latest
docker run --platform linux/amd64 nginx
```

Q104. What is Docker copy-on-write (CoW)?

Difficulty:
Medium

Copy-on-Write: storage optimization for image layers

How it works:

Base image layers: read-only

Container layer: writable, starts empty

Read file: look in container layer first, then image layers (top-down)

Write file: copy from image layer to container layer first, then modify

Delete file: create "whiteout" entry in container layer

Performance implications:

First write to file: slow (copy from lower layer)

Subsequent writes: fast (already in container layer)

Large file write: copies entire file even if modifying 1 byte

Best practice:

Large files that change frequently → use volumes (bypass CoW)

Databases MUST use volumes (terrible CoW performance for DB files)

Build artifacts → volumes or bind mounts in dev

overlay2 specifics:

lowerdir: read-only image layers (merged with overlay)

upperdir: writable container layer

workdir: overlay internal use

merged: unified view (what container sees)

Q105. What is graceful shutdown in Docker containers?

Difficulty:
Medium

```
// Docker stop: sends SIGTERM, waits 10s, then SIGKILL
```

```
// docker stop -t 30 container → 30s grace period
```

```
// Handle SIGTERM in Go
```

```
func main() {
    server := &http.Server{Addr: ":8080"}

    go server.ListenAndServe()

    quit := make(chan os.Signal, 1)
    signal.Notify(quit, syscall.SIGTERM, syscall.SIGINT)
    <-quit

    log.Println("shutting down...")
    ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
    defer cancel()

    if err := server.Shutdown(ctx); err != nil {
        log.Fatal("forced shutdown:", err)
    }
    log.Println("server stopped cleanly")
}
```



```
# Docker configuration
STOPSIGNAL SIGTERM      # default signal (can change to SIGQUIT)

# Compose
stop_grace_period: 30s  # wait 30s before SIGKILL
stop_signal: SIGTERM

# Important: if PID 1 doesn't handle SIGTERM → immediate SIGKILL
# Use tini: ENTRYPOINT ["/sbin/tini", "--"]
```

Q106. What are Docker labels?

Difficulty:
Medium

```
# Labels: metadata for images and containers
LABEL maintainer="team@example.com"
LABEL version="1.0.0"
LABEL description="My application"
LABEL org.opencontainers.image.title="MyApp"
LABEL org.opencontainers.image.version="1.0.0"
LABEL org.opencontainers.image.revision="${GIT_COMMIT}"
LABEL org.opencontainers.image.created="${BUILD_DATE}"
LABEL org.opencontainers.image.source="https://github.com/org/repo"

# Build with dynamic labels
docker build \
  --label "git-commit=$(git rev-parse --short HEAD)" \
  --label "build-date=$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
  -t myapp:latest .

# Filter/inspect by label
docker ps --filter label=env=production
docker images --filter label=version=1.0.0
docker inspect --format='{{json .Config.Labels}}' myimage
```

Q107. What is Docker overlay network internals?

Difficulty:
Medium

Overlay network (Docker Swarm): multi-host container networking

Components:

- VXLAN (Virtual Extensible LAN): tunnels Layer 2 over Layer 3 (UDP 4789)
- Gossip protocol: node discovery and network state sync
- Ingress network: routes external traffic to service replicas

Packet flow (container on Node A → container on Node B):

1. Container A sends to overlay network gateway
2. Overlay driver encapsulates in VXLAN UDP packet
3. Packet sent to Node B's IP
4. Node B decapsulates VXLAN
5. Delivers to destination container

Encryption:

- docker network create --opt encrypted myoverlay
- Uses IPSec AH (Authentication Header) for encryption

IPvlan vs Macvlan:

- Macvlan: container has own MAC (L2 switch behavior)

IPvlan L2: shared MAC, own IP
 IPvlan L3: routing between containers (no broadcasts)

Q108. What is Docker for microservices service mesh?

Difficulty:
Medium

Service mesh (Istio, Linkerd, Consul Connect):

Handles: mTLS, traffic routing, retries, circuit breaking, observability
 Deployed as: sidecar containers (envoy proxy alongside each service)

Without service mesh:

Each service implements: retry, timeout, circuit breaker, mTLS
 Duplicated logic in every service

With service mesh (Istio):

Sidecar proxy handles: encryption, retries, load balancing
 Application just makes plain HTTP calls
 Control plane (istiod) configures all proxies

Docker Compose + service mesh (dev):

```
services:
  app:
    image: myapp
  app-sidecar:
    image: envoyproxy/envoy:v1.28
    volumes:
      - ./envoy.yaml:/etc/envoy/envoy.yaml
    network_mode: service:app # same network namespace as app
```

Kubernetes: service mesh standard

Istio: most features (complex)
 Linkerd: lightweight, Go-based
 Consul Connect: HashiCorp ecosystem

Q109. What is Docker image provenance and SLSA?

Difficulty:
Medium

SLSA (Supply chain Levels for Software Artifacts):

Framework for supply chain security levels 1-4

Docker image provenance:

- WHO built the image (identity)
- WHAT source code (git commit)
- HOW it was built (build system)
- WHEN it was built (timestamp)

Tools:

SLSA GitHub Generator: creates SLSA attestations in CI
 cosign: signs and verifies image signatures
 Syft: generates SBOM (Software Bill of Materials)
 Grype: vulnerability scanner for SBOMs

GitHub Actions example:

```
- uses: slsa-framework/slsa-github-generator/.github/workflows/builder_go_slsa3.yml
  with:
    go-version: "1.22"
```

Verify:

```
cosign verify-attestation --type slsaprovenance myimage:tag
slsa-verifier verify-image myimage:tag \
  --source-uri github.com/org/repo \
  --source-tag v1.0.0
```

Q110. What is Docker Compose watch vs air (live reload)?

Difficulty:
Medium

```
// Development live reload options:

// 1. Air (Go hot reload)
// .air.toml
[build]
  cmd = "go build -o ./tmp/main ."
  bin = "./tmp/main"
  include_ext = ["go", "tpl", "tmpl", "html"]
  exclude_dir = ["vendor", "testdata"]

// docker-compose.yml with air
services:
  app:
    build:
      context: .
      target: dev
    volumes:
      - ./app
    command: air -c .air.toml

// 2. Docker Compose watch (Docker 2.22+)
services:
  app:
    develop:
      watch:
        - path: ./cmd
          action: rebuild
        - path: ./internal
          action: rebuild
        - path: ./config
          action: sync+restart
          target: /app/config

// 3. CompileDaemon
command: ["CompileDaemon", "--build=go build -o app .", "--command=./app"]

// Air is most popular for Go development
// Provides: instant rebuild, colored output, configurable
```

Q111. What is Docker BuildKit advanced features?

Difficulty:
Medium

```
# BuildKit: next-gen build engine (default since Docker 23.0)

# Secret mount: pass secrets without embedding in image
# --secret id=mysecret,src=./secret.txt
RUN --mount=type=secret,id=mysecret \
  cat /run/secrets/mysecret && \
  npm install --registry $(cat /run/secrets/mysecret)
```

```
# SSH mount: use host SSH keys inside build
# docker build --ssh default .
RUN --mount=type=ssh git clone git@github.com:private/repo.git

# Cache mount: persist between builds
RUN --mount=type=cache,target=/root/.npm npm ci
RUN --mount=type=cache,target=/go/pkg/mod go mod download
RUN --mount=type=cache,target=/root/.cache/pip pip install -r requirements.txt

# Heredoc syntax (Docker 1.4+ with BuildKit)
RUN <<EOF
apt-get update
apt-get install -y curl git
rm -rf /var/lib/apt/lists/*
EOF

# Build with BuildKit:
DOCKER_BUILDKIT=1 docker build .
# Or: docker buildx build . (always uses BuildKit)
```

Q112. What is Docker multi-platform builds?

Difficulty:
Medium

```
# Build for multiple CPU architectures
docker buildx create --name builder --use
docker buildx inspect --bootstrap

# Build for linux/amd64 and linux/arm64
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  --tag myapp:latest \
  --push \ # push manifest list to registry
.

# Build only for current platform (local testing)
docker buildx build --platform linux/amd64 --load .

# Emulation (QEMU): build arm64 on amd64 host
docker run --privileged --rm tonistiigi/binfmt --install all

# In Dockerfile: use TARGETPLATFORM
FROM golang:1.22 AS builder
ARG TARGETPLATFORM
ARG TARGETOS
ARG TARGETARCH
RUN GOOS=${TARGETOS} GOARCH=${TARGETARCH} go build -o app .

# CI: GitHub Actions
- uses: docker/setup-qemu-action@v3 # QEMU for emulation
- uses: docker/setup-buildx-action@v3
- uses: docker/build-push-action@v5
  with:
    platforms: linux/amd64,linux/arm64
    push: true
    tags: myapp:latest
```

Q113. What is Docker layer caching strategy?

Difficulty:
Medium

```
# Layer caching: each RUN/COPY/ADD creates a layer
# Cache invalidated when: instruction changes, or parent layer changes

# BAD: copy everything first → any file change invalidates all layers
COPY . .
RUN go mod download # cache miss if any .go file changes!

# GOOD: dependency files first (change rarely)
COPY go.mod go.sum ./
RUN go mod download      # cached unless go.mod/go.sum changes
COPY . .                 # copy source (often changes)
RUN go build -o app .     # only rebuilds binary

# Order of COPY: least-changing to most-changing
COPY package.json package-lock.json ./ # change: only on dep update
RUN npm ci                             # cached
COPY public/ ./public/                 # change: occasionally
COPY src/ ./src/                       # change: frequently

# BuildKit cache mounts (cross-build caching):
RUN --mount=type=cache,target=/go/pkg/mod,sharing=locked \
    --mount=type=cache,target=/root/.cache/go-build,sharing=locked \
    go build -o app .
# Cache persists between builds even if go.mod changes!

# GitHub Actions: cache Docker layers
- uses: docker/build-push-action@v5
  with:
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

Q114. What is Docker resource constraints?

Difficulty:
Medium

```
# docker-compose.yml resource limits
services:
  api:
    image: myapp:latest
    deploy:
      resources:
        limits:
          cpus: "2.0" # max 2 CPU cores
          memory: 512M # max 512MB RAM
        reservations:
          cpus: "0.5" # guaranteed 0.5 CPU
          memory: 128M # guaranteed 128MB RAM

# docker run:
docker run \
  --cpus="2.0" \           # limit to 2 CPUs
  --memory="512m" \       # max 512MB RAM
  --memory-swap="512m" \  # no swap (swap = memory limit)
  --memory-reservation="128m" \ # soft limit
  --cpu-shares=512 \      # relative weight (default 1024)
```

```
myapp:latest

# OOM Killer: if container exceeds memory → process killed
# Check: docker inspect container | grep OOMKilled
# GOMEMLIMIT (Go): prevent Go heap from reaching hard limit
# GOMAXPROCS (Go): set to container CPU limit (use uber-go/automaxprocs)

# automaxprocs: reads cgroup CPU quota
import _ "go.uber.org/automaxprocs"
// Automatically sets GOMAXPROCS based on cgroup limits
```

Q115. What is Docker image tagging strategy?

Difficulty:
Medium

```
# Tagging strategies for production:

# 1. Semantic versioning (recommended)
docker build -t myapp:1.2.3 .
docker tag myapp:1.2.3 myapp:1.2
docker tag myapp:1.2.3 myapp:1
docker tag myapp:1.2.3 myapp:latest

# 2. Git commit SHA (immutable, traceable)
GIT_SHA=$(git rev-parse --short HEAD)
docker build -t myapp:${GIT_SHA} .
docker build -t myapp:main-${GIT_SHA} .

# 3. Combined (best of both)
docker build \
  -t myapp:1.2.3 \
  -t myapp:1.2.3-abc1234 \ # version + commit
  -t registry.example.com/myapp:1.2.3 .

# 4. Date-based (useful for nightly builds)
docker build -t myapp:$(date +%Y%m%d) .

# Immutable tags policy:
# NEVER overwrite a tag that has been deployed to production
# ALWAYS use commit SHA for production deployments
# latest: only for local development convenience

# Registry: use Docker Hub, ECR, GCR, or self-hosted (Harbor)
# Retention: clean up old images (ecr lifecycle policy)
aws ecr put-lifecycle-policy --registry-id 123 --repository-name myapp \
  --lifecycle-policy-text '{"rules":[{"rulePriority":1,"selection":{"tagStatus":"untagged","countType":"since"}}

```

Q116. What is Docker secrets management?

Difficulty:
Medium

```
# docker-compose with secrets (swarm mode)
version: "3.8"
secrets:
  db_password:
    external: true # created via: docker secret create db_password ./password.txt

services:
  app:
```

```

    image: myapp:latest
    secrets:
      - db_password
    environment:
      - DB_PASSWORD_FILE=/run/secrets/db_password # path, not value!

# In application: read from file
password, _ := os.ReadFile("/run/secrets/db_password")

# For non-Swarm (docker-compose without swarm):
# Use .env file (not committed to git)
# Use environment variables from CI/CD (GitHub Secrets, etc.)
# Use Vault Agent sidecar

# External secrets management:
# 1. HashiCorp Vault: dynamic secrets, rotation
# 2. AWS Secrets Manager: managed, rotation built-in
# 3. Kubernetes Secrets (base64, not encrypted by default)
# 4. sealed-secrets: encrypted in git (decrypted by controller)

# NEVER: store secrets in Dockerfile or image layers
# docker history image:tag → shows all ENV variables!
# ENV DB_PASSWORD=secret → visible to anyone with image access!

```

Q117. What is Docker container networking deep dive?

Difficulty:
Medium

```

# Network drivers:
# bridge: default, isolated network with NAT
# host: share host network namespace (no NAT overhead)
# none: no networking
# overlay: multi-host (Swarm mode)
# macvlan: container gets its own MAC/IP on LAN

# Bridge network (default):
# docker0: virtual bridge, 172.17.0.0/16
# Container gets: 172.17.0.2, 172.17.0.3, etc.
# NAT: host IP → container IP (iptables MASQUERADE)

# Custom bridge (better than default):
docker network create \
  --driver bridge \
  --subnet 172.20.0.0/24 \
  --gateway 172.20.0.1 \
  mynet

# Containers on same custom bridge: can resolve by name (DNS)
docker run --network mynet --name db postgres
docker run --network mynet --name app myapp
# app → db:5432 (resolves by container name!)

# Port publishing (host → container):
-p 8080:80 # host:container
-p 127.0.0.1:8080:80 # bind to localhost only (security!)
--expose 80 # expose to other containers (no host binding)

# Host network (lowest latency, no NAT):

```

```
docker run --network host myapp
# Container uses host's IP directly
# Ports 80 → literally host port 80
# Use for: high-throughput services where NAT overhead matters
```

Q118. What is Docker log management?

Difficulty:
Medium

```
# Logging drivers
services:
  app:
    image: myapp
    logging:
      driver: json-file          # default
      options:
        max-size: "10m"         # rotate at 10MB
        max-file: "3"           # keep 3 rotated files

    logging:
      driver: fluentd            # ship to Fluentd
      options:
        fluentd-address: localhost:24224
        tag: myapp

    logging:
      driver: awslogs            # ship to CloudWatch
      options:
        awslogs-group: /app/myapp
        awslogs-region: us-east-1
        awslogs-stream: container-id

# Best practice: stdout/stderr in container → log driver ships logs
# Application should NOT write to files inside container!

# View logs:
docker logs container-name      # all logs
docker logs -f container-name   # follow
docker logs --since 1h container-name # last hour
docker logs --tail 100 container-name # last 100 lines

# Log format: JSON with timestamps
{"log":"Started server on :8080\n","stream":"stdout","time":"2024-01-15T10:30:00Z"}

# Structured logging in app → machine parseable by log aggregators
# slog.Info("request", "method", "GET", "path", "/api", "duration_ms", 12)
```

Q119. What is Docker Swarm vs Kubernetes?

Difficulty:
Medium

Docker Swarm:

- Simple setup (docker swarm init)
- Built into Docker (no extra tools)
- Simple YAML (docker-compose compatible)
- Good for: small teams, simple deployments
- Limited ecosystem
- Less features than Kubernetes
- Smaller community (declining)

- No advanced scheduling, RBAC, CRDs

Kubernetes:

- Industry standard (massive ecosystem)
- Advanced: autoscaling, RBAC, CRDs, operators
- Managed options: EKS, GKE, AKS (ops burden reduced)
- Service mesh (Istio, Linkerd)
- GitOps (ArgoCD, Flux)
- Complex setup and operations
- Steep learning curve
- Overkill for simple apps

Decision:

1-5 services, simple team → Docker Compose (local) + single server
 5-50 services, growing team → Kubernetes (managed: EKS/GKE)
 Enterprise → Kubernetes with platform team

Migration:

Docker Compose → Kubernetes: kompose convert (converts compose to K8s manifests)
 docker-compose.yml → k8s yamls

Q120. What is Docker Content Trust and image signing?

Difficulty:
Medium

```
# Docker Content Trust (DCT): sign and verify images
# Uses Notary (The Update Framework)

# Enable DCT (blocks pulling unsigned images)
export DOCKER_CONTENT_TRUST=1

# Sign and push image
docker trust key generate mykey
docker trust signer add --key mykey.pub myuser docker.io/myorg/myapp
docker push docker.io/myorg/myapp:1.0.0 # auto-signed with DCT=1

# Verify signature
docker trust inspect --pretty docker.io/myorg/myapp:1.0.0

# Modern alternative: cosign (sigstore)
# Keyless signing using OIDC (GitHub Actions, etc.)
cosign sign --yes myapp:latest
cosign verify --certificate-identity-regexp="github.com/myorg" \
  --certificate-oidc-issuer="https://token.actions.githubusercontent.com" \
  myapp:latest

# GitHub Actions: sign with OIDC (no key management!)
- uses: sigstore/cosign-installer@v3
- name: Sign image
  run: cosign sign --yes ${env.REGISTRY}/${env.IMAGE_NAME}:${env.VERSION}

# Verify in Kubernetes (Kyverno or OPA Gatekeeper):
# Policy: only allow images signed by our CI pipeline
```

Q121. What is Docker healthcheck vs Kubernetes probes?

Difficulty:
Medium

```
# Docker HEALTHCHECK: built-in container health monitoring
```

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=30s --retries=3 \
  CMD curl -f http://localhost:8080/healthz || exit 1

# States: starting, healthy, unhealthy
# Docker Swarm: restarts unhealthy containers
# Docker standalone: informational only (no auto-restart)

# In docker-compose:
services:
  app:
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/healthz"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 30s
    depends_on:
      db:
        condition: service_healthy

# Kubernetes: liveness + readiness + startup probes
spec:
  containers:
  - name: app
    livenessProbe:      # restart if failing (is it alive?)
      httpGet: {path: /healthz, port: 8080}
      initialDelaySeconds: 30
      periodSeconds: 10
      failureThreshold: 3
    readinessProbe:     # remove from LB if failing (is it ready?)
      httpGet: {path: /readyz, port: 8080}
      periodSeconds: 5
      failureThreshold: 3
    startupProbe:       # extra time for slow-starting containers
      httpGet: {path: /healthz, port: 8080}
      failureThreshold: 30 # 30 * 10s = 5 min startup window
      periodSeconds: 10
```

Q122. What is Docker rootless mode?

Difficulty:
Medium

```
# Rootless Docker: run Docker daemon as non-root user
# Security benefit: container breakout → only user privileges (not root!)

# Install rootless Docker
dockerd-rootless-setuptool.sh install

# Run rootless Docker
export DOCKER_HOST=unix://$XDG_RUNTIME_DIR/docker.sock
docker run hello-world

# Limitations:
# No privileged containers
# No --network host (requires root for privileged ports)
# No cgroup v1 resource limits (use cgroup v2)
# Some performance overhead (user namespaces)

# Podman: rootless by default!
```

```
# Drop-in Docker replacement, rootless, no daemon
podman run nginx
podman pod create --name mypod
podman generate kube mypod > mypod.yaml # generate K8s YAML!
```

```
# rootless in Kubernetes:
# runAsNonRoot: true
# runAsUser: 1000
# allowPrivilegeEscalation: false
# readOnlyRootFilesystem: true
```

Q123. What is Docker entrypoint vs CMD?

Difficulty:
Medium

```
# ENTRYPOINT: fixed executable (not overridable without --entrypoint)
# CMD: default arguments (overridable by docker run arguments)

# Pattern 1: ENTRYPOINT + CMD for default + overridable args
ENTRYPOINT ["/app/server"]      # always run server
CMD ["--port=8080"]             # default port, overridable

docker run myapp                # runs: /app/server --port=8080
docker run myapp --port=9090    # runs: /app/server --port=9090

# Pattern 2: CMD only (common for dev images)
CMD ["/app/server", "--port=8080"]
docker run myapp bash           # runs: bash (overrides CMD completely!)

# Pattern 3: Shell script as ENTRYPOINT (init tasks)
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
ENTRYPOINT ["/entrypoint.sh"]
CMD ["server"]

# entrypoint.sh:
#!/bin/sh
set -e
# Run migrations
/app/migrate up
# Then run the command
exec "$@" # exec: replace shell with command (PID 1 = app)
# exec is critical: app receives SIGTERM, not shell

# Exec form vs Shell form:
# Exec: ["cmd", "arg1"] → PID 1 is cmd (receives signals)
# Shell form: cmd arg1 → PID 1 is /bin/sh (signals not forwarded!)
```

Q124. What is Docker image layers and size optimization?

Difficulty:
Medium

```
# Analyze image layers
docker history myapp:latest
docker image inspect myapp:latest

# dive: interactive layer explorer
dive myapp:latest # shows: layer size, wasted space, efficiency score

# Common size issues:
```

```
# 1. Build tools left in final image
FROM golang:1.22 AS builder    # includes Go toolchain (~500MB)
FROM scratch AS final         # empty (0MB)
COPY --from=builder /app/server /server # just the binary (~10MB)

# 2. Cache files not cleaned
# BAD:
RUN apt-get update && apt-get install -y curl
# GOOD:
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*

# 3. Multiple RUN commands (each creates layer)
# BAD:
RUN apt-get update
RUN apt-get install -y curl
RUN apt-get install -y git
# GOOD:
RUN apt-get update && apt-get install -y --no-install-recommends curl git \
    && rm -rf /var/lib/apt/lists/*

# 4. .dockerignore missing
echo "node_modules\n.git\n*.log\n.env\ntmp/" > .dockerignore

# Size targets:
# Go microservice: 10-20MB (scratch/distroless)
# Node.js: 100-200MB (node:alpine)
# Python: 100-300MB (python:slim)
```

Q125. What is Docker compose profiles?

Difficulty:
Medium

```
# Profiles: selectively start services
version: "3.8"
services:
  app:
    image: myapp
    # no profile = always started

  db:
    image: postgres:16
    profiles: [app, testing] # started with: docker compose --profile app up

  redis:
    image: redis:7
    profiles: [app]

  test-runner:
    image: myapp-test
    profiles: [testing] # only for: docker compose --profile testing up

  prometheus:
    image: prom/prometheus
    profiles: [monitoring] # optional: docker compose --profile monitoring up

# Usage:
docker compose up                # only app (no profile)
```

```
docker compose --profile app up          # app + db + redis
docker compose --profile testing up      # app + db + test-runner
docker compose --profile app --profile monitoring up # multiple profiles

# Use case:
# dev: full stack (app + db + redis)
# ci: app + test services
# monitoring: add prometheus + grafana
# integration: add mock external services
```

Q126. What is Docker ARG vs ENV?

Difficulty:
Medium

```
# ARG: build-time variable (not in final image, not in containers)
# ENV: runtime variable (in final image, accessible in container)
```

```
ARG GO_VERSION=1.22
FROM golang:${GO_VERSION}    # use ARG in FROM

ARG BUILD_DATE
ARG GIT_COMMIT
LABEL build-date=${BUILD_DATE}
LABEL git-commit=${GIT_COMMIT}

# Build with ARG values:
docker build \
  --build-arg GO_VERSION=1.22 \
  --build-arg GIT_COMMIT=$(git rev-parse --short HEAD) \
  --build-arg BUILD_DATE=$(date -u +%Y-%m-%dT%H:%M:%SZ) \
  .

# ENV: available at runtime
ENV APP_ENV=production
ENV PORT=8080
# Override at runtime: docker run -e PORT=9090 myapp

# Security:
# ARG values visible in docker history (don't use for secrets!)
# ENV values visible in docker inspect (don't use for secrets!)
# SECRETS: use --secret (BuildKit) or runtime injection

# Multi-stage: ARG scoped to stage
FROM builder AS final
ARG VERSION # must re-declare in each stage that uses it
ENV APP_VERSION=${VERSION} # persist to ENV for runtime
```

Q127. What is Docker Compose wait-for dependencies?

Difficulty:
Medium

```
# Problem: app starts before db is ready → connection error!

services:
  app:
    image: myapp
    depends_on:
      db:
        condition: service_healthy    # wait for healthcheck to pass
```

```

    redis:
      condition: service_started # just wait for container to start

db:
  image: postgres:16
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    timeout: 5s
    retries: 10
    start_period: 10s

redis:
  image: redis:7
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s
    retries: 5

// Alternative: retry in application (more resilient)
func connectWithRetry(ctx context.Context, dsn string) (*pgxpool.Pool, error) {
    for i := 0; i < 30; i++ {
        pool, err := pgxpool.New(ctx, dsn)
        if err == nil {
            if err = pool.Ping(ctx); err == nil {
                return pool, nil
            }
        }
        pool.Close()
    }
    log.Printf("waiting for DB... attempt %d/30", i+1)
    time.Sleep(2 * time.Second)
}

return nil, errors.New("could not connect to DB after 30 attempts")
}

```

Q128. What is Docker registry and image distribution?

Difficulty:
Medium

```

# Registry: stores Docker images (layers + manifests)
# Protocol: Docker Registry HTTP API V2

# Self-hosted registries:
# Docker Registry (official, basic)
docker run -d -p 5000:5000 --name registry registry:2
docker push localhost:5000/myapp:latest

# Harbor (enterprise features: vulnerability scan, RBAC, replication)
# GHCR (GitHub Container Registry): free for public repos
# ECR (AWS): tight IAM integration
# GCR (Google): GCP integration

# Push workflow:
docker login registry.example.com
docker build -t registry.example.com/myorg/myapp:1.0.0 .
docker push registry.example.com/myorg/myapp:1.0.0

# Pull with auth
docker pull registry.example.com/myorg/myapp:1.0.0

```

```
# Image manifest (multi-arch):
docker manifest inspect nginx:latest

# Layer sharing:
# Layers identified by content hash (sha256)
# Same layer in 10 images → stored once → significant storage savings
# Push: only uploads new layers (existing layers skipped)

# Garbage collection: remove unreferenced layers
# (manual in open-source registry, automatic in managed services)
```

Q129. What is Docker compose override files?

Difficulty:
Medium

```
# docker-compose.yml (base, committed to git)
services:
  app:
    image: myapp:latest
    environment:
      - APP_ENV=production

# docker-compose.override.yml (auto-merged, gitignored for local overrides)
services:
  app:
    build: .          # build instead of pull (dev override)
    volumes:
      - ./app         # live code reload
    environment:
      - APP_ENV=development
      - DEBUG=true

# docker-compose.prod.yml (explicit production overrides)
services:
  app:
    deploy:
      replicas: 3
    environment:
      - APP_ENV=production

# Override precedence (later files win):
docker compose up          # base + override.yml (automatic)
docker compose -f docker-compose.yml \
  -f docker-compose.prod.yml up  # explicit order

# Per-environment pattern:
docker-compose.yml          # base
docker-compose.override.yml # local dev (gitignored)
docker-compose.ci.yml       # CI overrides
docker-compose.prod.yml     # production overrides
```

Q130. What is Docker for CI/CD pipeline?

Difficulty:
Medium

```
# GitHub Actions: Docker build + push
name: Build and Push
on: [push]
jobs:
  build:
```

```

runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4

  - uses: docker/setup-buildx-action@v3

  - uses: docker/login-action@v3
    with:
      registry: ghcr.io
      username: ${ github.actor }
      password: ${ secrets.GITHUB_TOKEN }

  - uses: docker/metadata-action@v5
    id: meta
    with:
      images: ghcr.io/${ github.repository }
      tags: |
        type=semver,pattern={{version}}
        type=sha,prefix=,suffix=,format=short

  - uses: docker/build-push-action@v5
    with:
      context: .
      push: true
      tags: ${ steps.meta.outputs.tags }
      cache-from: type=gha
      cache-to: type=gha,mode=max
      platforms: linux/amd64,linux/arm64
      build-args: |
        GIT_COMMIT=${ github.sha }
        BUILD_DATE=${ github.event.head_commit.timestamp }

```

Q131. ### Q131-Q150: Final Docker Questions

Difficulty:
Medium

Q	Topic
Q131	Docker compose networking (service discovery)
Q132	Container startup ordering best practices
Q133	Docker volume types (bind, volume, tmpfs)
Q134	Docker image vulnerability scanning (trivy)
Q135	Docker compose environment variable files
Q136	Container restart policies and failure handling
Q137	Docker desktop alternatives (Podman, Orbstack)
Q138	Docker layer diffing and change tracking
Q139	Container runtime security (seccomp, AppArmor)
Q140	Docker compose project isolation
Q141	Multi-stage builds for testing + production
Q142	Docker image promotion across environments
Q143	Container metrics with cAdvisor

Q144	Docker Swarm rolling updates
Q145	Registry mirror and pull-through cache
Q146	Docker compose v2 vs v3 differences
Q147	Container debugging techniques
Q148	Docker CPU pinning and NUMA
Q149	Docker compose secrets (non-swarm)
Q150	Docker production readiness final checklist

Q132. What is Docker volume types comparison?

Difficulty:
Medium

```
# 1. Named volumes (managed by Docker, best for production)
docker volume create mydata
docker run -v mydata:/data myapp
# Stored: /var/lib/docker/volumes/mydata/_data
# Pros: portable, managed by Docker, easy backup
# Use: databases, persistent app state

# 2. Bind mounts (host path mounted into container)
docker run -v /host/path:/container/path myapp
docker run -v $(pwd):/app myapp # current directory
# Pros: direct host access, live reload in dev
# Cons: host-dependent, less portable
# Use: development, config files

# 3. tmpfs mounts (in-memory, not persisted)
docker run --tmpfs /tmp:rw,size=100m myapp
# Use: sensitive data (secrets, tokens), temp files
# Docker Compose:
services:
  app:
    tmpfs:
      - /tmp:size=100m
      - /run

# Volume in Compose:
services:
  db:
    volumes:
      - pgdata:/var/lib/postgresql/data # named volume
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql # bind
volumes:
  pgdata: # declare named volume
```

Q133. What is Docker trivy vulnerability scanning?

Difficulty:
Medium

```
# trivy: fast, comprehensive vulnerability scanner
# Scans: OS packages, language deps (go, npm, pip, etc.)

# Install
brew install trivy # macOS
# or: docker run aquasec/trivy
```

```
# Scan image
trivy image myapp:latest
trivy image --severity HIGH,CRITICAL myapp:latest
trivy image --exit-code 1 --severity CRITICAL myapp:latest # fail CI on CRITICAL

# Scan local filesystem
trivy fs .
trivy fs --security-checks vuln,config .

# Scan IaC (Dockerfile, docker-compose, K8s manifests)
trivy config .

# SBOM generation
trivy image --format cyclonedx --output sbom.json myapp:latest
trivy image --format spdx-json --output sbom.json myapp:latest

# CI integration (GitHub Actions)
- uses: aquasecurity/trivy-action@master
  with:
    image-ref: myapp:latest
    format: sarif
    output: trivy-results.sarif
    severity: CRITICAL,HIGH
    exit-code: 1
```

Q134. What is Docker compose environment configuration?

Difficulty:
Medium

```
# .env file (auto-loaded by docker compose)
# .env
DB_HOST=postgres
DB_PORT=5432
DB_PASSWORD=secret
APP_VERSION=1.2.3

# docker-compose.yml
services:
  app:
    image: myapp:${APP_VERSION} # variable substitution
    environment:
      - DB_HOST=${DB_HOST} # from .env
      - DB_PORT=${DB_PORT:-5432} # with default
      - APP_ENV # pass-through from shell
    env_file:
      - .env # explicit file
      - .env.local # override (gitignored)

# Multiple env files (later overrides earlier):
docker compose --env-file .env.prod up

# Shell variables override .env:
DB_PASSWORD=override docker compose up

# Validation: fail if required variable not set
services:
  app:
    environment:
```

- REQUIRED_VAR=\${REQUIRED_VAR:?Error: REQUIRED_VAR is not set}

Q135. What is Docker container lifecycle?

Difficulty:
Medium

```
# States: created → running → paused | stopped | dead

# Create (doesn't start)
docker create --name myapp myapp:latest

# Start
docker start myapp

# Run (create + start + attach)
docker run --name myapp myapp:latest

# Pause (SIGSTOP to all processes)
docker pause myapp
docker unpause myapp

# Stop (SIGTERM → wait → SIGKILL)
docker stop myapp          # 10s timeout
docker stop -t 30 myapp    # 30s timeout

# Kill (SIGKILL immediately)
docker kill myapp
docker kill --signal SIGTERM myapp # specific signal

# Remove
docker rm myapp          # stopped container
docker rm -f myapp       # force remove running container
docker rm $(docker ps -aq) # remove all stopped containers

# Restart policies:
# no:          never restart (default)
# always:      always restart
# on-failure:3: restart on failure, max 3 times
# unless-stopped: restart unless manually stopped

docker run --restart=unless-stopped myapp
```

Q136. What is Docker compose networking?

Difficulty:
Medium

```
# By default: Compose creates one network (project_default)
# All services join it and can communicate by service name

services:
  app:
    # can reach: db:5432, redis:6379, nginx:80
  db:
    image: postgres
  redis:
    image: redis
  nginx:
    image: nginx

# Custom networks for isolation
```

```

services:
  frontend:
    networks: [frontend]
  backend:
    networks: [frontend, backend] # bridge between networks
  db:
    networks: [backend]          # not reachable from frontend!
  redis:
    networks: [backend]

networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
    internal: true # no internet access (only internal)

# External network (shared across Compose projects)
networks:
  shared:
    external: true # created with: docker network create shared

```

Q137. What is Docker debugging techniques?

Difficulty:
Medium

```

# Enter running container
docker exec -it container bash
docker exec -it container sh # alpine/distroless (no bash)

# Debug distroless/scratch containers (no shell)
# Method 1: ephemeral debug container
docker debug container-name # Docker Desktop 4.27+
# or: kubectl debug -it pod --image=busybox --target=app

# Method 2: docker cp + manual inspection
docker cp container:/app/logs/error.log .

# Method 3: nsenter into process namespace
container_pid=$(docker inspect --format '{{.State.Pid}}' container)
nsenter -t $container_pid -m -u -i -n -p bash

# Inspect state
docker inspect container          # full JSON config
docker stats container            # real-time resource usage
docker top container              # processes running
docker diff container             # filesystem changes vs image
docker port container             # port mappings

# Copy files in/out
docker cp ./config.yaml container:/app/config.yaml
docker cp container:/app/dump.json .

# Run one-off debug container with same network
docker run --rm -it --network container:myapp \
  nicolaka/netshoot curl http://localhost:8080/healthz

```

Q138. What is Docker security capabilities?

Difficulty:
Medium

```
# Linux capabilities: fine-grained privileges (alternative to full root)
# Default dropped by Docker: many dangerous capabilities
# Never use: --privileged (gives ALL capabilities + host device access)

# View capabilities a container has
docker run --rm ubuntu capsh --print

# Drop ALL then add only what's needed (principle of least privilege)
docker run \
  --cap-drop ALL \
  --cap-add NET_BIND_SERVICE \ # bind ports < 1024
  --cap-add CHOWN \           # chown files
  nginx

# Common capabilities:
# NET_BIND_SERVICE: bind ports < 1024 (needed for nginx on port 80)
# CHOWN:           change file owner
# DAC_OVERRIDE:    bypass file permission checks
# SYS_PTRACE:      trace processes (for debugging, never in prod)
# SYS_ADMIN:       mount filesystems, sethostname (avoid!)
# NET_ADMIN:       configure network interfaces (avoid!)

# Seccomp profile (block syscalls)
docker run --security-opt seccomp=default.json myapp

# AppArmor (restrict filesystem/network access)
docker run --security-opt apparmor=docker-default myapp

# No new privileges (prevent privilege escalation)
docker run --security-opt no-new-privileges myapp
```

Q139. What is Docker swarm services?

Difficulty:
Medium

```
# Swarm: Docker native clustering (manager + worker nodes)
docker swarm init # init swarm on manager
docker swarm join --token TOKEN IP:PORT # join worker

# Deploy service
docker service create \
  --name web \
  --replicas 3 \
  --publish 80:80 \
  --update-delay 10s \
  --update-parallelism 2 \
  --rollback-parallelism 2 \
  nginx:latest

# Scale
docker service scale web=5

# Rolling update
docker service update \
  --image nginx:1.25 \
```

```
--update-delay 10s \
--update-parallelism 1 \ # one at a time
web

# Rollback if update goes wrong
docker service rollback web

# Stack (multi-service, like compose)
docker stack deploy -c docker-compose.yml mystack
docker stack ls
docker stack services mystack
docker stack ps mystack # tasks (containers)
docker stack rm mystack

# Service discovery: VIP (Virtual IP) load balancing
# DNS: web → resolves to VIP → routes to any replica
# DNSRR mode: returns all replica IPs (client-side LB)
```

Q140. What is Docker image build cache invalidation?

Difficulty:
Medium

```
# Cache key = instruction content + parent layer hash

# This invalidates ALL subsequent layers:
# - Any file changed in COPY/ADD
# - Any RUN command text changed
# - ENV/ARG value changed
# - Base image updated (FROM)

# Strategy: most stable layers first, most volatile last
FROM golang:1.22-alpine

# Layer 1: base tools (changes: never)
RUN apk add --no-cache git ca-certificates

# Layer 2: dependencies (changes: occasionally)
COPY go.mod go.sum ./
RUN go mod download

# Layer 3: source code (changes: often)
COPY . .
RUN go build -o app .

# Force cache bust (useful when RUN script changes but text doesn't)
ARG CACHEBUST=1
RUN --mount=type=cache,target=/tmp/cache,id=bust-${CACHEBUST} ./install.sh

# Build with cache bust:
docker build --build-arg CACHEBUST=$(date +%s) .

# Check if layer was cached:
docker build --progress=plain . 2>&1 | grep "CACHED"
```

Q141. What is Docker compose profiles for dev vs prod?

Difficulty:
Medium

```
version: "3.8"
```

```

services:
  # Core services (always run)
  app:
    image: myapp:${TAG:-latest}
    environment:
      - DATABASE_URL=postgres://user:pass@db:5432/mydb

  db:
    image: postgres:16
    volumes:
      - pgdata:/var/lib/postgresql/data

# Development only
mailhog:
  image: mailhog/mailhog
  profiles: [dev]
  ports: ["8025:8025"] # email UI

seed:
  image: myapp:${TAG:-latest}
  command: ./seed-db
  profiles: [dev]
  depends_on: [db]

# Monitoring (optional)
prometheus:
  image: prom/prometheus
  profiles: [monitoring]
  volumes: ["/etc/prometheus/prometheus.yml"]

grafana:
  image: grafana/grafana
  profiles: [monitoring]

volumes:
  pgdata:

# Run with profiles
docker compose --profile dev up          # app + db + mailhog + seed
docker compose --profile monitoring up    # app + db + prometheus + grafana
docker compose up                         # just app + db

```

Q142. ### Q142–Q150: Docker Final Questions

Difficulty:
Medium

Q	Topic
Q142	Docker image promotion pipeline
Q143	Container resource limits in Kubernetes
Q144	Docker compose dependency ordering
Q145	Registry garbage collection
Q146	Docker compose watch mode for live reload
Q147	Container DNS resolution
Q148	Docker save/load for air-gapped environments

Q149	Container init systems (tini, dumb-init)
Q150	Docker complete production checklist

Q143. What is Docker image promotion pipeline?

Difficulty:
Medium

```
# Image promotion: same image progresses through environments
# Key: build ONCE, promote everywhere (don't rebuild per env)

# Build once with SHA tag
GIT_SHA=$(git rev-parse --short HEAD)
docker build -t registry.example.com/myapp:${GIT_SHA} .
docker push registry.example.com/myapp:${GIT_SHA}

# Promote to staging (tag + push, no rebuild)
docker pull registry.example.com/myapp:${GIT_SHA}
docker tag registry.example.com/myapp:${GIT_SHA} \
    registry.example.com/myapp:staging
docker push registry.example.com/myapp:staging

# Promote to production (same image, new tag)
docker tag registry.example.com/myapp:${GIT_SHA} \
    registry.example.com/myapp:v1.2.3
docker tag registry.example.com/myapp:${GIT_SHA} \
    registry.example.com/myapp:production
docker push registry.example.com/myapp:v1.2.3
docker push registry.example.com/myapp:production

# Benefits:
# - dev = staging = prod (same binary)
# - Env config injected at runtime via env vars
# - Audit: SHA tag traces back to exact commit
# - Rollback: re-tag previous SHA to production

# Env-specific config (NOT baked in):
# Kubernetes: ConfigMap + Secret injected as env vars
# Docker: -e DATABASE_URL=prod-db-host
```

Q144. What is container resource limits in Kubernetes?

Difficulty:
Medium

```
# Requests: guaranteed resources (used for scheduling)
# Limits: max resources (enforced by kernel)

apiVersion: v1
kind: Pod
spec:
  containers:
  - name: app
    image: myapp:latest
    resources:
      requests:
        memory: "128Mi" # scheduler places pod on node with >= 128Mi free
        cpu: "250m" # 0.25 CPU cores requested
      limits:
        memory: "512Mi" # OOM killed if exceeds (no swap in K8s)
```



```

    cpu: "1000m"          # 1 CPU core max (throttled, not killed)

# CPU throttling vs OOM:
# CPU limit exceeded → throttled (process slows down)
# Memory limit exceeded → OOM killed (container restarted)

# QoS classes:
# Guaranteed: requests == limits (highest priority, not evicted first)
# Burstable:  requests < limits (medium priority)
# BestEffort: no requests/limits (evicted first under pressure)

# Go: set GOMAXPROCS to match CPU request
import _ "go.uber.org/automaxprocs"
# Reads /sys/fs/cgroup/cpu.cfs_quota_us → sets GOMAXPROCS

# GOMEMLIMIT: set to 90% of memory limit
# GOMEMLIMIT=460MiB (for 512Mi limit)

```

Q145. What is Docker compose dependency ordering?

Difficulty:
Medium

```

# depends_on: controls start order
# Does NOT wait for service to be "ready" (only container started)
# For readiness: use healthchecks + condition: service_healthy

services:
  app:
    depends_on:
      db:
        condition: service_healthy      # wait for healthcheck PASS
      redis:
        condition: service_started      # just wait for container start
      migrator:
        condition: service_completed_successfully # wait for exit 0

  db:
    image: postgres:16
    healthcheck:
      test: ["CMD-SHELL", "pg_isready"]
      interval: 5s
      retries: 10
      start_period: 10s

  migrator:
    image: myapp:latest
    command: ["/migrate", "up"]
    depends_on:
      db:
        condition: service_healthy
    restart: "no" # run once, don't restart

  redis:
    image: redis:7
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]

# Startup order: db → redis → migrator → app

```

migrator must succeed (exit 0) before app starts

Q146. What is Docker registry garbage collection?

Difficulty:
Medium

Problem: pushing images accumulates unreferenced layers → disk fills up
Garbage collection: remove blobs not referenced by any manifest

```
# Docker Registry v2: built-in GC
# Enable delete API in config.yml:
storage:
  delete:
    enabled: true
```

```
# Run GC (mark phase: find unreferenced blobs)
docker exec registry /bin/registry garbage-collect \
  /etc/docker/registry/config.yml
```

```
# Dry run first (see what would be deleted)
docker exec registry /bin/registry garbage-collect \
  --dry-run /etc/docker/registry/config.yml
```

```
# WARNING: registry must be read-only during GC
# (or use --delete-untagged flag on newer versions)
```

```
# Harbor: automated GC with retention policies
# Admin → Garbage Collection → Schedule
# Retention policies: keep last N tags, keep tags matching pattern
```

```
# AWS ECR: lifecycle policies (recommended)
aws ecr put-lifecycle-policy \
  --repository-name myapp \
  --lifecycle-policy-text '{
    "rules": [{
      "rulePriority": 1,
      "selection": {
        "tagStatus": "untagged",
        "countType": "sinceImagePushed",
        "countUnit": "days",
        "countNumber": 7
      },
      "action": {"type": "expire"}
    }],
  }'
```

Q147. What is Docker compose watch mode?

Difficulty:
Medium

```
# Docker Compose watch (v2.22+): file change → auto rebuild/sync
services:
  app:
    build: .
    develop:
      watch:
        # Sync source code without rebuild (for interpreted languages)
        - path: ./src
          action: sync
```

```

    target: /app/src
    ignore:
      - node_modules/

# Rebuild image on dependency changes
- path: ./package.json
  action: rebuild
- path: ./go.mod
  action: rebuild

# Restart (sync + restart without full rebuild)
- path: ./config
  action: sync+restart
  target: /app/config

# Start with watch mode
docker compose watch

# Comparison with volume mounts:
# Volume bind mount: instant sync, no rebuild needed
# Docker watch: suitable for compiled languages (Go, Java)
#                   rebuild triggered only on key file changes
#                   cleaner than always-on volume mounts

# For Go development: prefer air (hot reload):
# docker run -v ./app cosmtrek/air

```

Q148. What is Docker container DNS resolution?

Difficulty:
Medium

```

# Within Docker bridge network: containers resolve by service name
# DNS server: 127.0.0.11 (Docker embedded DNS resolver)

# Verify DNS inside container
docker exec myapp cat /etc/resolv.conf
# nameserver 127.0.0.11
# options ndots:0

# Resolve another service
docker exec myapp nslookup db
# Server: 127.0.0.11
# Address: 172.20.0.2 (db container IP)

# Custom DNS server
docker run --dns 8.8.8.8 myapp
docker run --dns-search example.com myapp

# Docker Compose: adds container name + service name as aliases
# If service name = "database", network alias "database" resolves to it
# Also: container name (if unique) resolves

# Host aliases (for testing)
services:
  app:
    extra_hosts:
      - "api.external.com:192.168.1.100" # /etc/hosts override
      - "host.docker.internal:host-gateway" # access host machine

```

```
# host.docker.internal: resolve host IP from container
# Works on: Docker Desktop (Mac/Windows)
# Linux: needs --add-host=host.docker.internal:host-gateway
```

Q149. What is Docker save/load for air-gapped environments?

Difficulty:
Medium

```
# Air-gapped: no internet access (banks, government, high-security)
# Solution: export images to tar files, transfer via approved channels
```

```
# Save single image to tar
docker save myapp:1.2.3 -o myapp-1.2.3.tar
# With compression:
docker save myapp:1.2.3 | gzip > myapp-1.2.3.tar.gz
```

```
# Save multiple images (single tar file)
docker save myapp:1.2.3 postgres:16 redis:7 | gzip > stack.tar.gz
```

```
# Load from tar (on air-gapped machine)
docker load -i myapp-1.2.3.tar
docker load < myapp-1.2.3.tar.gz # with gzip
```

```
# Transfer options:
# - Encrypted USB drive
# - Approved file transfer system
# - Secure copy (scp) to internal server
```

```
# Sign before transfer (verify integrity)
sha256sum myapp-1.2.3.tar.gz > myapp-1.2.3.tar.gz.sha256
# Verify on receiving end:
sha256sum -c myapp-1.2.3.tar.gz.sha256
```

```
# Skopeo: copy between registries (alternative to save/load)
# skopeo copy docker://source-registry/myapp:1.2.3 \
#           docker://internal-registry/myapp:1.2.3
# Supports: docker, oci, docker-archive formats
```

Q150. What is container init systems (tini, dumb-init)?

Difficulty:
Medium

```
# Problem: PID 1 in container must handle signals properly
# Many apps don't: shell scripts, Python apps, Java apps
# Result: SIGTERM ignored → Docker must SIGKILL after timeout
```

```
# tini: minimal init process for containers
FROM ubuntu:22.04
RUN apt-get install -y tini
ENTRYPOINT ["/usr/bin/tini", "--"]
CMD ["/app/server"]
```

```
# Or via Docker --init flag (uses tini built into Docker)
docker run --init myapp
```

```
# What tini does:
# 1. Forwards signals to child process
# 2. Reaps zombie processes (wait() on orphaned children)
# 3. Exits with child's exit code
```

```
# dumb-init: alternative to tini (Yelp)
FROM ubuntu:22.04
RUN apt-get install -y dumb-init
ENTRYPOINT ["/usr/bin/dumb-init", "--"]

# Go apps: usually don't need tini
# Go handles signals natively, no zombie issue
# Python/Node: benefit from tini
# Shell scripts (!/bin/sh): use exec to replace shell
# ENTRYPOINT ["/entrypoint.sh"]
# entrypoint.sh: exec "$@" ← replaces shell with app (PID 1 = app)

# Verify PID 1
docker exec mycontainer ps aux | head -5
# PID 1 should be tini/your app, not /bin/sh
```

Q151. What is the Docker complete production checklist?

Difficulty:
Medium

Image:

- Multi-stage build (minimal final image)
- Non-root user (USER nonroot:nonroot)
- Specific base image tag pinned (not :latest)
- .dockerignore configured (exclude .git, node_modules, .env)
- No secrets in image (Dockerfile, ENV, ARG)
- Vulnerability scan passing (trivy image → no CRITICAL)
- Image size optimized (< 50MB for Go, < 300MB for Node)
- ENTRYPOINT exec form (["cmd", "arg"] not "cmd arg")
- HEALTHCHECK defined

Runtime:

- Memory limits set (--memory)
- CPU limits set (--cpus)
- Restart policy: unless-stopped
- No --privileged mode
- Capabilities dropped (--cap-drop ALL + add only needed)
- Read-only filesystem where possible (--read-only)
- Tmpfs for writable temp dirs

Networking:

- Ports bound to 127.0.0.1 if not externally needed
- Internal services on isolated networks
- TLS termination at reverse proxy

Logging:

- Structured logging (JSON to stdout)
- Log rotation (max-size, max-file)
- Centralized log aggregation

Observability:

- Prometheus metrics exposed
- Health endpoint (/healthz, /readyz)
- Resource usage monitored (cAdvisor)

Secrets:

- Secrets injected at runtime (env vars or file mounts)
- .env files gitignored

